

09:00-09:50 (쉬는 시간 11:30-12:00)

오디오 파일 계약과 메타데이터

Codec · Container · audio_metadata.json

진행

오디오 파일 계약과 메타데이터
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/demo_meeting.wav
data/demo_meeting_transcript.txt

확인할 결과

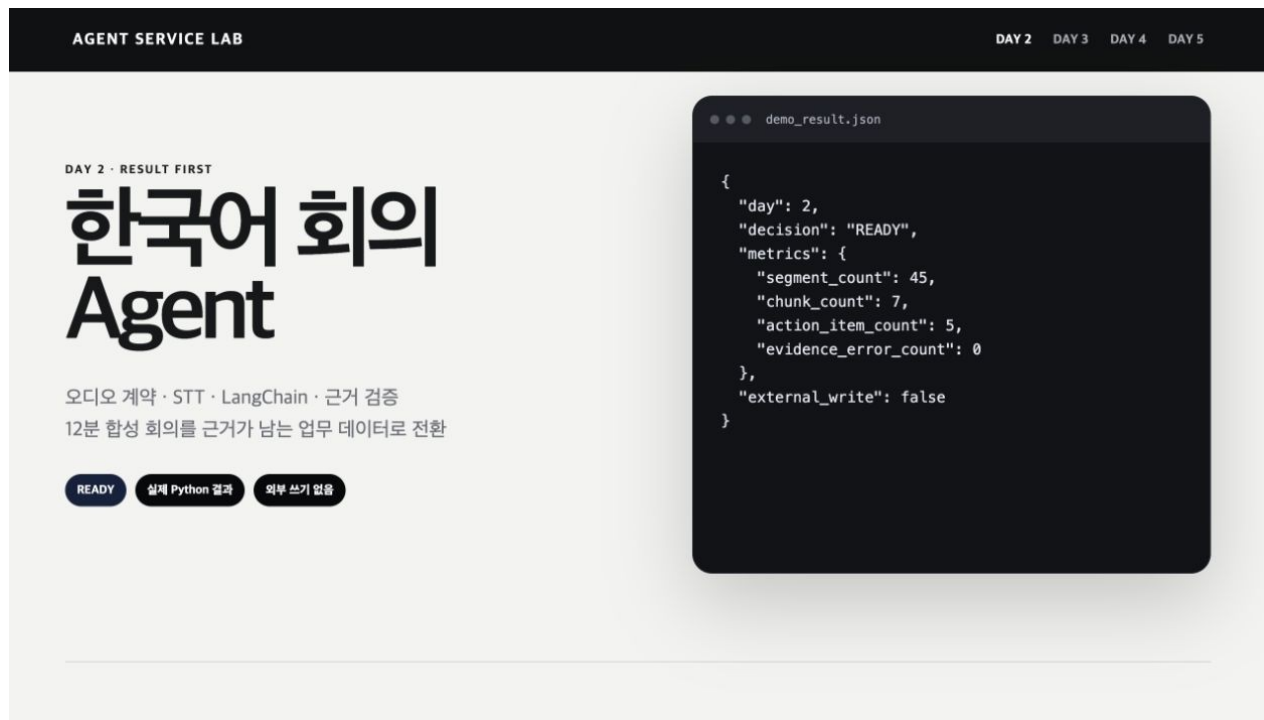
audio_metadata.json

Meeting Intelligence Service 완성 화면

깨진 입력을 모델 문제로 오해하면 복구
순서가 뒤집힌다.

입력 계약

- 핵심 처리
- 실패 경계
- 사람 판단
- 평가·실행 증거



재생 파일 · assets/demo-videos/day2_service_teaser.gif

오늘 8차시는 이 화면을 현재 저장소에서 다시 만드는 과정입니다.

2일차 전체 시간표

3시간 블록(30분 휴식) · 2시간 블록(20분 휴식) · 점심 14:00-15:00 · 3시간 블록(30분 휴식)

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	오디오 파일 계약과 메타데이터	강의 12 시연 10 Codex 제작 23 확인 5분	audio_metadata.json
09:50-10:40	2차시	faster-whisper 로컬 STT Adapter	강의 12 시연 10 Codex 제작 23 확인 5분	transcript.json
10:40-11:30	3차시	STT 품질 Gate	강의 12 시연 10 Codex 제작 23 확인 5분	quality_gate.json
12:00-12:50	4차시	회의록 Schema와 Pydantic 계약	강의 12 시연 10 Codex 제작 23 확인 5분	meeting_schema.json
12:50-13:40	5차시	발화 단위 Chunking	강의 12 시연 10 Codex 제작 23 확인 5분	meeting_chunks.json
15:00-15:50	6차시	LangChain MeetingBrief Pipeline	강의 12 시연 10 Codex 제작 23 확인 5분	meeting_brief.json
15:50-16:40	7차시	Action Item 근거 검증	강의 12 시연 10 Codex 제작 23 확인 5분	validated_meeting_brief.json
16:40-17:30	8차시	도메인별 Golden Set	강의 12 시연 10 Codex 제작 23 확인 5분	day2_eval_cases.json

11:30-12:00 쉬는 시간 · 13:40-14:00 쉬는 시간 · 14:00-15:00 점심시간 · 17:30-18:00 쉬는 시간

오디오 파일 계약과 메타데이터 학습 결과

- 01 — 길이·채널·샘플링레이트를 읽고 전사 가능 여부를 판단한다.
- 02 — 파일이 없거나 codec을 열 수 없는데 재시도만 반복한다. 왜 위험한지 설명한다.
- 03 — audio_metadata.json에서 정상과 실패 상태를 직접 확인한다.

오디오 파일 계약과 메타데이터 용어

용어	이 수업에서 쓰는 뜻
Codec	음성을 저장하는 압축 방식
Container	음성과 메타데이터를 담는 파일 형식
Sample rate	1초를 몇 번 측정했는지
Channel	mono·stereo 같은 음성 통로 수

audio_metadata.json 입출력

INPUT

data/demo_meeting.wav

OUTPUT

audio_metadata.json

PROCESS

workspace 경로 확인 → 파일 존재 확인 → WAV metadata 읽기

VERIFY

WAV의 길이·rate·channel이 숫자로 반환된다.
workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

오디오 파일 계약과 메타데이터 실행 흐름

01

workspace 경로
확인

02

파일 존재 확인

03

WAV metadata 읽기

04

지원 조건 판정

05

전사 또는 구조화 실패

마지막 판단은 audio_metadata.json에서 사람이 확인합니다.

audio_metadata.json 정상·실패 계약

정상	——	WAV의 길이·rate·channel이 숫자로 반환된다.
경계	——	workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
외부 쓰기	——	오디오 파일 계약과 메타데이터: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·audio_metadata.json

오디오 파일 계약과 메타데이터 파일 지도

입력·fixture

data/demo_meeting.wav

구현 코드

data/demo_meeting_transcript.txt

검증·test

materials/day2/day2_service_lab.ipynb

따라하기 notebook

src/meeting_demo.py

오디오 파일 계약과 메타데이터 개념·코드

Codec

음성을 저장하는 압축 방식

```
data/demo_meeting.wav
```

Container

음성과 메타데이터를 담는 파일 형식

```
data/demo_meeting_transcript.txt
```

Sample rate

1초를 몇 번 측정했는지

```
materials/day2/day2_service_lab.ipynb
```

Channel

mono·stereo 같은 음성 통로 수

```
src/meeting_demo.py
```

오디오 파일 계약과 메타데이터 선택 기준

구분	역할	이번 차시의 판단 기준
Codec	workspace 경로 확인 단계의 입력 기준	결정론 test로 먼저 확인
Container	파일 존재 확인 단계의 교체 경계	adapter 경계를 확인
Sample rate	WAV metadata 읽기 단계의 운영 조건	비용·지연·권한을 확인
Channel	지원 조건 판정 단계의 판단 기록	사람 판단과 운영 기록을 확인

오디오 파일 계약과 메타데이터 대표 실패

실패

파일이 없거나 codec을 열 수 없는데
재시도만 반복한다.

사용자 영향

파일이 없거나 codec을 열 수 없는데 재시도만
반복한다. 상태에서는 audio_metadata.json을
운영 판단에 사용할 수 없다.

오디오 파일 계약과 메타데이터 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

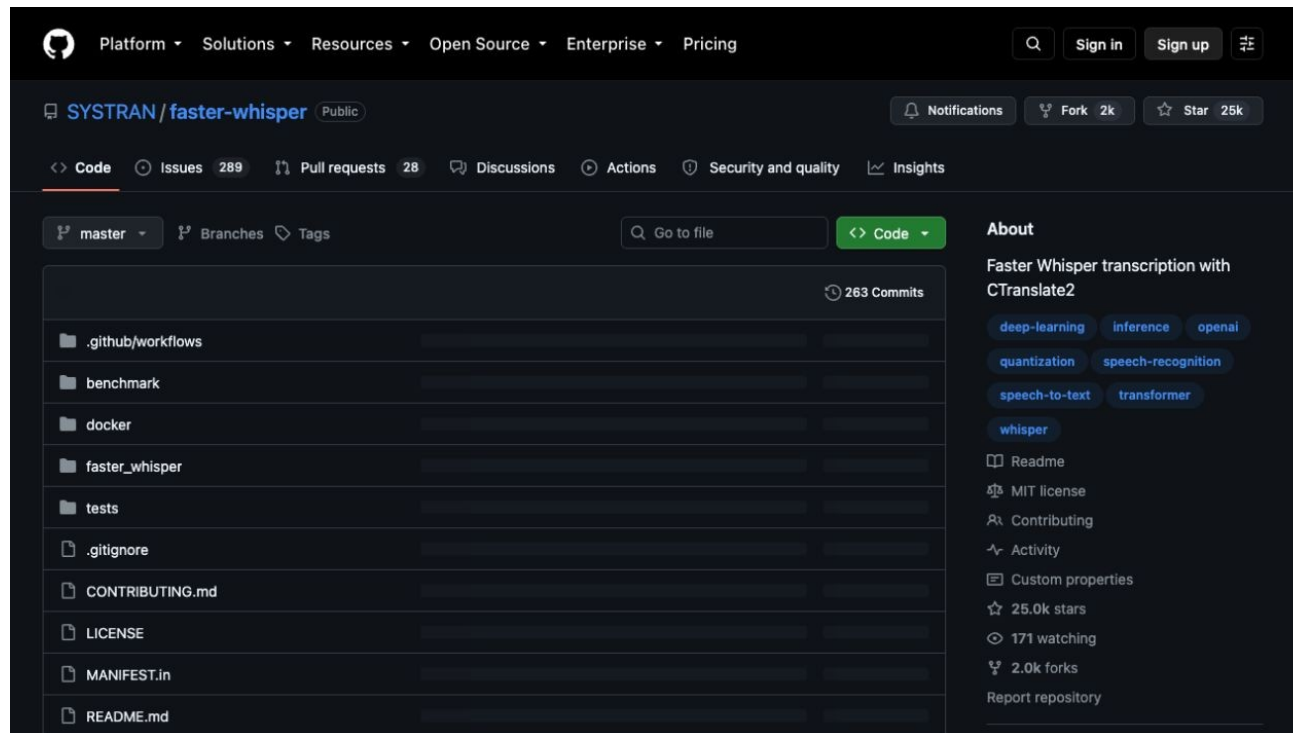
오디오 파일 계약과 메타데이터 복구 경로

- 01 — 탐지: workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.
- 02 — 중단: 파일이 없거나 codec을 열 수 없는데 재시도만 반복한다.
- 03 — 복구: FILE_NOT_FOUND와 UNSUPPORTED_AUDIO를 구분하고 변환 또는 fixture 경로를 안내한다.
- 04 — 증거: audio_metadata.json과 test 결과를 함께 남긴다.

오디오 파일 계약과 메타데이터 Codex 검증 루프

data/
demo_meeting_transcript.txt의
책임을 찾고 audio_metadata.json
계약을 focused test로 고정합니다.

오디오 파일 계약과 메타데이터
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



오디오 파일 계약과 메타데이터 Codex 작업 명세

목표

오디오 metadata 검사 함수를 작은 책임으로 분리하고 정상·경로 이탈 test를 추가한다.

허용 경로

data/demo_meeting_transcript.txt
materials/day2/day2_service_lab.ipynb

완료 조건

WAV의 길이·rate·channel이 숫자로 반환된다.
workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

금지

오디오 파일 계약과 메타데이터: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

오디오 파일 계약과 메타데이터 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | 오디오 파일 계약과 메타데이터 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | WAV의 길이·rate·channel이 숫자로 반환된다. |
| 03 | Claude review | —— | audio_metadata.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 오디오 파일 계약과 메타데이터의 두 LLM 의견과 test는 자동 승인이 아니다 |

오디오 파일 계약과 메타데이터 정상 시연

- 01 — 열기: `data/demo_meeting.wav`
- 02 — 구현 확인: `data/demo_meeting_transcript.txt`
- 03 — 실행: `python -m src.meeting_demo --help`
- 04 — 성공 신호: WAV의 길이·rate·channel이 숫자로 반환된다.

오디오 파일 계약과 메타데이터 실행 명령

```
$ python -m src.meeting_demo --help
```

실행 전 확인

오디오 파일 계약과 메타데이터: repository root · Python 3.12 · .env 미출력

01_audio_metadata.json 실행 결과

```
{  
  "status": "SUCCESS",  
  "seconds": 1017.46,  
  "sample_rate": 22050,  
  "channels": 1,  
  "sample_width": 2  
}
```

오디오 파일 계약과 메타데이터 실패·복구 시연

재현

workspace 밖 오디오는
WORKSPACE_PATH_BLOCKED로
끝난다.

복구

FILE_NOT_FOUND와
UNSUPPORTED_AUDIO를 구분하고
변환 또는 fixture 경로를 안내한다.

확인: workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

오디오 파일 계약과 메타데이터 소프트웨어 제작

열 파일

`data/demo_meeting_transcript.txt`
`materials/day2/day2_service_lab.ipynb`

작업 범위

오디오 파일 계약과 메타데이터 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

WAV의 길이·rate·channel이 숫자로 반환된다.
`audio_metadata.json` 저장

오디오 파일 계약과 메타데이터 Notebook 셀

materials/day2/day2_service_lab.ipynb · 1차시

```
import wave
from src.meeting_demo import ensure_workspace_path

audio_path = ensure_workspace_path(ROOT / "data/meeting_sample_ko_12min.wav", ROOT)
with wave.open(str(audio_path), "rb") as wav:
    audio_metadata = {
        "status": "SUCCESS",
        "seconds": round(wav.getnframes() / wav.getframerate(), 2),
        "sample_rate": wav.getframerate(),
        "channels": wav.getnchannels(),
        "sample_width": wav.getsampwidth(),
    }
save_json("01_audio_metadata.json", audio_metadata)
audio_metadata
```

오디오 파일 계약과 메타데이터 Terminal 실행

```
$ python -m src.meeting_demo --help
```

저장 위치

```
output/course-labs/day2/01_audio_metadata.json
```

WAV의 길이·rate·channel이 숫자로 반환된다.
실패 시 audio_metadata.json의 status·error_code를 먼저 확인

오디오 파일 계약과 메타데이터 실패 증거

workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

```
result_file = "output/course-labs/day2/01_audio_metadata.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

audio_metadata.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

오디오 파일 계약과 메타데이터 정상 Case

NORMAL

WAV의 길이·rate·channel이 숫자로 반환된다.

- 01 — data/demo_meeting.wav 입력과 command가 기록됐는가
- 02 — audio_metadata.json schema가 validate되는가
- 03 — 오디오 파일 계약과 메타데이터 focused test가 실제로 통과했는가

오디오 파일 계약과 메타데이터 경계 Case

BOUNDARY

workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다.

- 01 — 오디오 파일 계약과 메타데이터: raw traceback 대신 stable error_code가 남는가
- 02 — audio_metadata.json: 외부 쓰기·자동 메일이 false인가
- 03 — workspace 밖 오디오는 WORKSPACE_PATH_BLOCKED로 끝난다. 재현이 가능한가

오디오 파일 계약과 메타데이터 현업 적용

고객 상담 녹음이 들어오기 전에 포맷과 보존 정책을 검사하는 업로드 단계

- 01 — 오디오 파일 계약과 메타데이터는 합성·비식별 sample로 먼저 실행
- 02 — 고객 상담 녹음이 들어오기 전에 포맷과 보존 정책을 검사하는 업로드 단계의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — audio_metadata.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — audio_metadata.json을 운영 검토 자료로 사용

오디오 파일 계약과 메타데이터 포트폴리오 적용

오디오 파일 계약과 메타데이터 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 파일이 없거나 codec을 열 수 없는데 재시도만 반복한다.의 사용자 영향을 한 문장으로 설명
- 02 ——— 오디오 파일 계약과 메타데이터 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— 오디오 metadata 검사 함수를 작은 책임으로 분리하고 정상·경로 이탈 test를 추가한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— audio_metadata.json과 README 재현 절차를 제출

오디오 파일 계약과 메타데이터 운영 점검

품질

무엇을 WAV의 길이·rate·channel이 숫자로 반환된다.로 정의하는가

복구

파일이 없거나 codec을 열 수 없는데 재시도만 반복한다.
발생 시 audio_metadata.json에서 원인 상태를 확인

안전

오디오 파일 계약과 메타데이터의 사람 승인과
external_write=false 위치

관측

audio_metadata.json에서 어떤 status·error_code를
보는가

audio_metadata.json 실행 증거

- 01 — 설명: 오디오 파일 계약과 메타데이터가 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_demo --help`
- 03 — 증거: `audio_metadata.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: **faster-whisper 로컬 STT Adapter**

DAY 2 · 2차시

09:50-10:40 (쉬는 시간 11:30-12:00)

faster-whisper 로컬 STT Adapter

Whisper · faster-whisper · transcript.json

진행

faster-whisper 로컬 STT Adapter
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

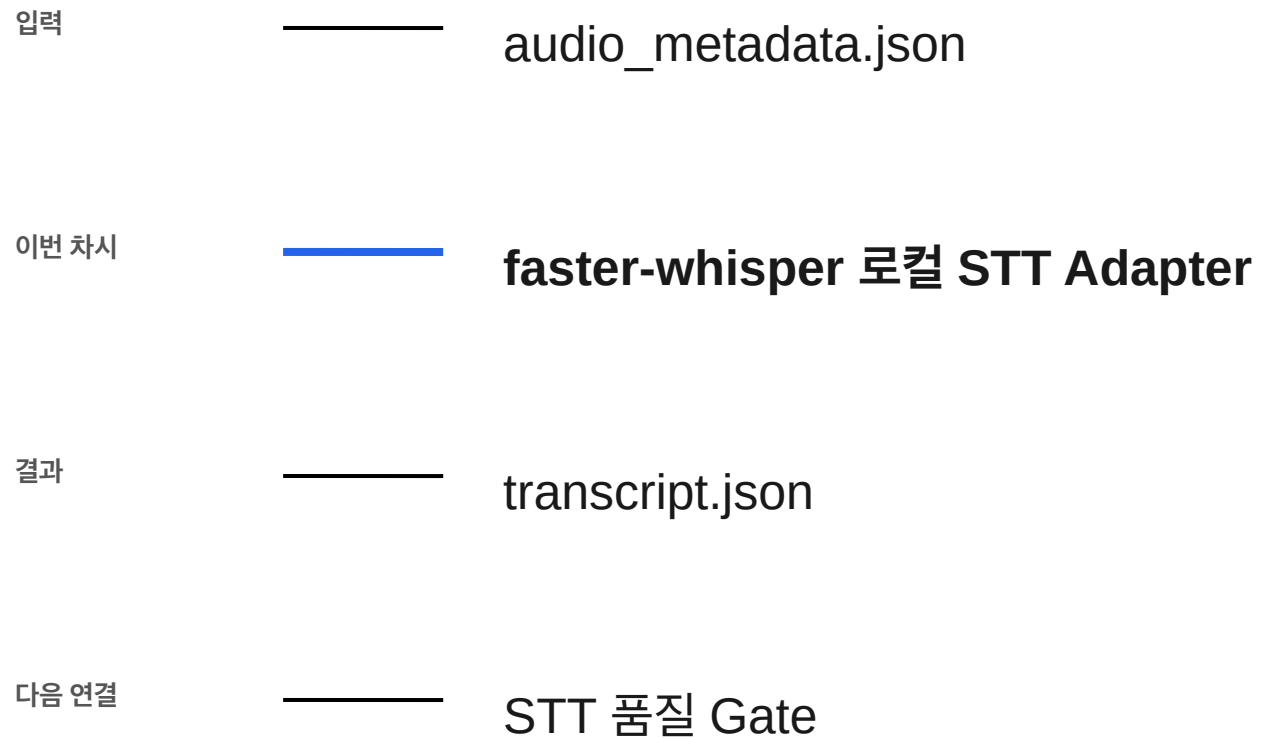
사용 파일

src/meeting_demo.py
requirements-stt-optional.txt

확인할 결과

transcript.json

2차시 입력과 결과



faster-whisper 로컬 STT Adapter 필요성

모델 호출과 나머지 업무 로직을 분리해야 GPU·모델·provider를 바꿀 수 있다.

모델 load에서 transcript.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: transcript.json

faster-whisper 로컬 STT Adapter 학습 결과

- 01 — CPU int8 환경에서 한국어 음성을 segment와 timestamp로 전사한다.
- 02 — 모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다. 왜 위험한지 설명한다.
- 03 — transcript.json에서 정상과 실패 상태를 직접 확인한다.

faster-whisper 로컬 STT Adapter 용어

용어	이 수업에서 쓰는 뜻
Whisper	음성을 문자로 바꾸는 모델 계열
faster-whisper	CTranslate2 기반 실행 구현
int8	메모리를 줄이는 계산 형식
Segment	시작·끝 시각이 붙은 발화 조각

transcript.json 입출력

INPUT

src/meeting_demo.py

OUTPUT

transcript.json

PROCESS

모델 load → VAD 적용 → beam search

VERIFY

segment마다 start·end·text가 있고 float가 JSON
직렬화된다.

STT package가 없으면 fixture와 STT_FALLBACK_USED
를 함께 남긴다.

faster-whisper 로컬 STT Adapter 실행 흐름

01

모델 load

02

VAD 적용

03

beam search

04

segment 순회

05

JSON 안전 float
변환

마지막 판단은 transcript.json에서 사람이 확인합니다.

transcript.json 정상·실패 계약

정상	——	segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
경계	——	STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
외부 쓰기	——	faster-whisper 로컬 STT Adapter: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·transcript.json

faster-whisper 로컬 STT Adapter 파일 지도

입력·fixture

src/meeting_demo.py

구현 코드

requirements-stt-optional.txt

검증·test

data/demo_meeting.wav

따라하기 notebook

output/day2-stt-lab/transcript.json

faster-whisper 로컬 STT Adapter 개념·코드

Whisper

음성을 문자로 바꾸는 모델 계열

```
src/meeting_demo.py
```

faster-whisper

CTranslate2 기반 실행 구현

```
requirements-stt-optional.txt
```

int8

메모리를 줄이는 계산 형식

```
data/demo_meeting.wav
```

Segment

시작·끝 시각이 붙은 발화 조각

```
output/day2-stt-lab/transcript.json
```


faster-whisper 로컬 STT Adapter 선택 기준

구분	역할	이번 차시의 판단 기준
Whisper	모델 load 단계의 입력 기준	결정론 test로 먼저 확인
faster-whisper	VAD 적용 단계의 교체 경계	adapter 경계를 확인
int8	beam search 단계의 운영 조건	비용·지연·권한을 확인
Segment	segment 순회 단계의 판단 기록	사람 판단과 운영 기록을 확인

faster-whisper 로컬 STT Adapter 대표 실패

실패

모델 download나 GPU OOM 때문에
후속 수업 전체가 멈춘다.

사용자 영향

모델 download나 GPU OOM 때문에 후속 수업
전체가 멈춘다. 상태에서는 transcript.json을
운영 판단에 사용할 수 없다.

faster-whisper 로컬 STT Adapter 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

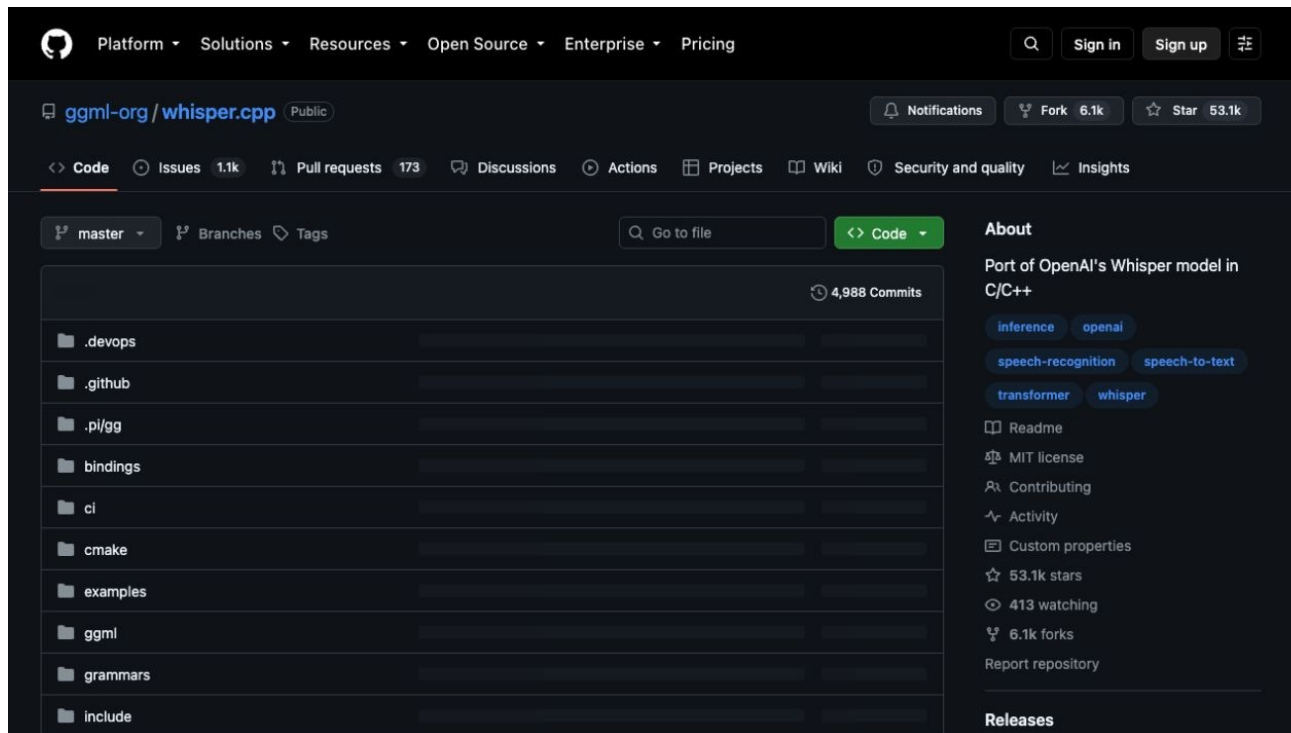
faster-whisper 로컬 STT Adapter 복구 경로

- 01 — 탐지: STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.
- 02 — 중단: 모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다.
- 03 — 복구: local_files_only·CPU int8·작은 모델·reviewed fixture 순서로 내려간다.
- 04 — 증거: transcript.json과 test 결과를 함께 남긴다.

faster-whisper 로컬 STT Adapter Codex 검증 루프

requirements-stt-optional.txt의 책임을 찾고 transcript.json 계약을 focused test로 고정합니다.

faster-whisper 로컬 STT Adapter
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



faster-whisper 로컬 STT Adapter Codex 작업 명세

목표

provider adapter가 같은 transcript contract를 반환하는지 test로 고정한다.

허용 경로

requirements-stt-optional.txt
data/demo_meeting.wav

완료 조건

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

금지

faster-whisper 로컬 STT Adapter: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

faster-whisper 로컬 STT Adapter 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | faster-whisper 로컬 STT Adapter 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | segment마다 start·end·text가 있고 float가 JSON 직렬화된다. |
| 03 | Claude review | —— | transcript.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | faster-whisper 로컬 STT Adapter의 두 LLM 의견과 test는 자동 승인이 아니다 |

faster-whisper 로컬 STT Adapter 정상 시연

- 01 — 열기: `src/meeting_demo.py`
- 02 — 구현 확인: `requirements-stt-optional.txt`
- 03 — 실행: `python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu --compute-type int8`
- 04 — 성공 신호: segment마다 `start·end·text`가 있고 `float`가 JSON 직렬화된다.

faster-whisper 로컬 STT Adapter 실행 명령

```
$ python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu  
--compute-type int8
```

실행 전 확인

faster-whisper 로컬 STT Adapter: repository root · Python 3.12 · .env 미출력

02_transcript.json 실행 결과

```
{
  "provider_used": "reviewed_fixture",
  "fallback_reason": "LIVE_STT_NOT_SELECTED",
  "segment_count": 45,
  "segments": [
    {
      "id": "s01",
      "start": 0,
      "speaker": "민지",
      "text": "모두 들어오셨죠? 오늘 회의 목적은 고객 문의 자동화 PoC의 첫 범위를 확정하는 것입니다. 모델을 고르는 회의가 아니라, 어떤 문의를 어디까지 자동화하고 어디서 사람이 확인할지  
정하는 회의로 진행하겠습니다.",
      "quality_flags": []
    },
    {
      "id": "s02",
      "start": 24,
      "speaker": "서연",
      "text": "운영 쪽 현황부터 말씀드릴게요. 최근 4주 동안 합성 기준으로 문의가 1,240건 정도였고, 배송 지연이 31퍼센트, 반품 절차가 24퍼센트, 환불과 보상 관련 문의가 18퍼센트였습니다.  
나머지는 계정, 쿠폰, 상품 정보처럼 유형이 조금씩 달랐습니다.",
      "quality_flags": []
    },
    {
      "id": "s03",
      "start": 49,
      "speaker": "준호",
      "text": "그중에서 API 없이도 답할 수 있는 범위를 먼저 골라야 할 것 같습니다. 배송 지연은 주문 상태 조회가 필요할 수 있고, 반품 절차는 정책 문서만으로도 초안 생성이 가능하니까 같은  
자동화로 묶으면 위험도가 다릅니다.",
      "quality_flags": []
    },
    ...
  ]
}
```

faster-whisper 로컬 STT Adapter 실패·복구 시연

재현

STT package가 없으면 fixture와
STT_FALLBACK_USED를 함께 남긴다.

복구

local_files_only·CPU int8·작은 모델
·reviewed fixture 순서로 내려간다.

확인: STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

faster-whisper 로컬 STT Adapter 소프트웨어 제작

열 파일	requirements-stt-optional.txt data/demo_meeting.wav
작업 범위	faster-whisper 로컬 STT Adapter 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	segment마다 start·end·text가 있고 float가 JSON 직렬화된다. transcript.json 저장

faster-whisper 로컬 STT Adapter Notebook 셀

materials/day2/day2_service_lab.ipynb · 2차시

```
from src.meeting_demo import parse_transcript, run_demo

transcript_path = ROOT / "data/meeting_sample_ko_12min.txt"
transcript_text = transcript_path.read_text(encoding="utf-8")
RUN_STT_LIVE = False
if RUN_STT_LIVE:
    stt_run = run_demo(
        audio_path=audio_path,
        transcript_path=transcript_path,
        output_dir=OUT / "live-stt",
        model_size="small", device="cpu", compute_type="int8",
        language="ko", local_files_only=True,
    )
    stt_segments = stt_run["segments"]
    stt_status = {"provider_used": stt_run["mode"], "fallback_reason": stt_run["fallback_reason"]}
else:
    stt_segments = parse_transcript(transcript_text)
    stt_status = {"provider_used": "reviewed_fixture", "fallback_reason": "LIVE_STT_NOT_SELECTED"}
transcript_result = {**stt_status, "segment_count": len(stt_segments), "segments": stt_segments}
save_json("02_transcript.json", transcript_result)
# ... 다음 줄은 Notebook에서 계속
```

faster-whisper 로컬 STT Adapter Terminal 실행

```
$ python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device  
cpu --compute-type int8
```

저장 위치

```
output/course-labs/day2/02_transcript.json
```

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.
실패 시 transcript.json의 status·error_code를 먼저 확인

faster-whisper 로컬 STT Adapter 실패 증거

STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

```
result_file = "output/course-labs/day2/02_transcript.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

transcript.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

faster-whisper 로컬 STT Adapter 정상 Case

NORMAL

segment마다 start·end·text가 있고 float가 JSON 직렬화된다.

- 01 — src/meeting_demo.py 입력과 command가 기록됐는가
- 02 — transcript.json schema가 validate되는가
- 03 — faster-whisper 로컬 STT Adapter focused test가 실제로 통과했는가

faster-whisper 로컬 STT Adapter 경계 Case

BOUNDARY

STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다.

- 01 — faster-whisper 로컬 STT Adapter: raw traceback 대신 stable error_code가 남는가
- 02 — transcript.json: 외부 쓰기·자동 메일이 false인가
- 03 — STT package가 없으면 fixture와 STT_FALLBACK_USED를 함께 남긴다. 재현이 가능한가

faster-whisper 로컬 STT Adapter 현업 적용

사내망에서 원본 음성을 외부로 보내지 않는 상담 품질 분석

- 01 — faster-whisper 로컬 STT Adapter은 합성·비식별 sample로 먼저 실행
- 02 — 사내망에서 원본 음성을 외부로 보내지 않는 상담 품질 분석의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — transcript.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — transcript.json을 운영 검토 자료로 사용

faster-whisper 로컬 STT Adapter 포트폴리오 적용

faster-whisper 로컬 STT Adapter · 문제·코드·실패·검증을 한 저장소로 제출

- 01 ——— 모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다.의 사용자 영향을 한 문장으로 설명
- 02 ——— faster-whisper 로컬 STT Adapter 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 ——— provider adapter가 같은 transcript contract를 반환하는지 test로 고정한다.의 diff와 직접 검토한 test 증거를 구분
- 04 ——— transcript.json과 README 재현 절차를 제출

faster-whisper 로컬 STT Adapter 운영 점검

품질

무엇을 segment마다 start·end·text가 있고 float가 JSON 직렬화된다.로 정의하는가

복구

모델 download나 GPU OOM 때문에 후속 수업 전체가 멈춘다. 발생 시 transcript.json에서 원인 상태를 확인

안전

faster-whisper 로컬 STT Adapter의 사람 승인과 external_write=false 위치

관측

transcript.json에서 어떤 status·error_code를 보는가

transcript.json 실행 증거

- 01 — 설명: faster-whisper 로컬 STT Adapter이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_demo --audio data/demo_meeting.wav --model small --device cpu --compute-type int8`
- 03 — 증거: transcript.json과 정상·실패 test 결과가 남아 있다.

다음 차시: STT 품질 Gate

DAY 2 · 3차시

10:40-11:30 (쉬는 시간 11:30-12:00)

STT 품질 Gate

VAD · no-speech · quality_gate.json

진행

STT 품질 Gate
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

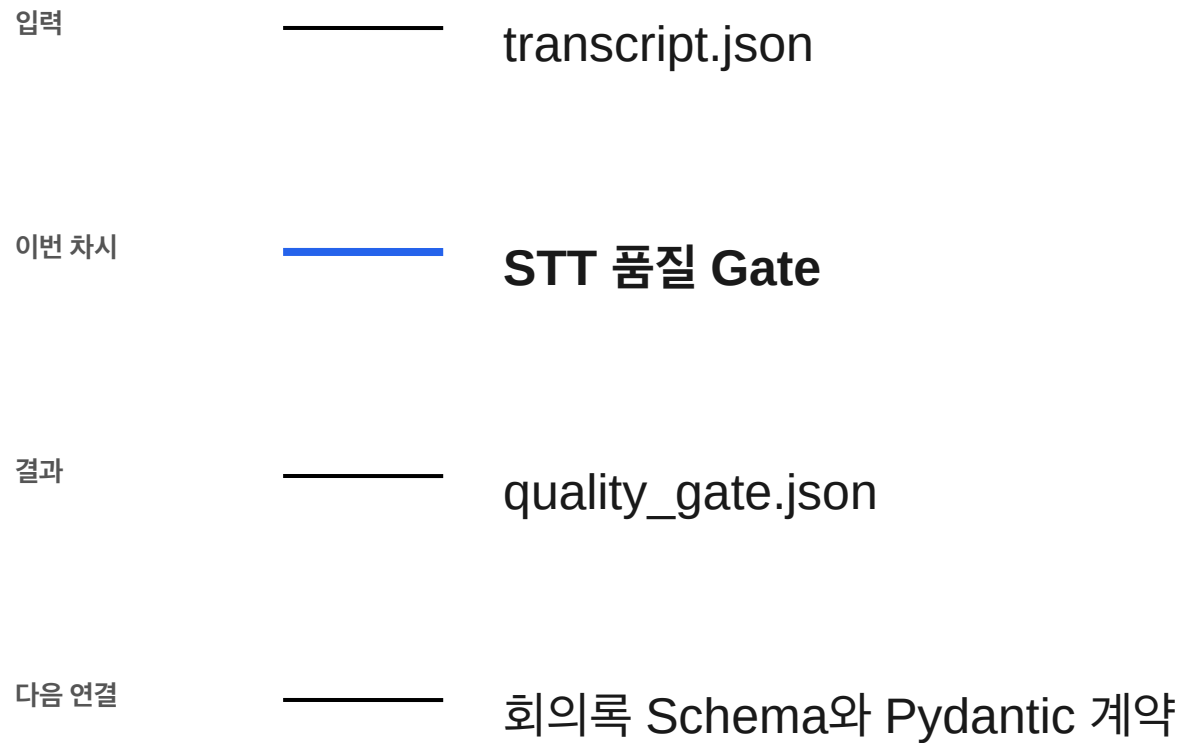
사용 파일

src/meeting_demo.py
tests/test_meeting_agent_workflow.py

확인할 결과

quality_gate.json

3차시 입력과 결과



STT 품질 Gate 필요성

문자가 생성됐다는 사실만으로 회의 결과를 만들면 이름과 숫자 오류가 그대로 번진다.

segment metric 수집에서 quality_gate.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: quality_gate.json

STT 품질 Gate 학습 결과

- 01 — 무음·반복·낮은 확률·언어 불일치를 품질 flag로 분리한다.
- 02 — 낮은 품질 segment 하나를 전체 성공으로 숨긴다. 왜 위험한지 설명한다.
- 03 — quality_gate.json에서 정상과 실패 상태를 직접 확인한다.

STT 품질 Gate 용어

용어	이 수업에서 쓰는 뜻
VAD	말소리 구간을 먼저 찾는 처리
no-speech	해당 구간이 무음일 가능성
log probability	모델이 전사에 가진 상대 신뢰
Hotwords	제품명 등 인식 힌트

quality_gate.json 입출력

INPUT

src/meeting_demo.py

OUTPUT

quality_gate.json

PROCESS

segment metric 수집 → 반복 탐지 → 언어 확인

VERIFY

한국어·충분한 길이·무 flag일 때 READY다.
fallback·짧은 전사·reference 유사도 저하는 HOLD다.

STT 품질 Gate 실행 흐름

01

segment metric
수집

02

반복 탐지

03

언어 확인

04

reference 비교

05

READY 또는 HOLD

마지막 판단은 `quality_gate.json`에서 사람이 확인합니다.

quality_gate.json 정상·실패 계약

정상	——	한국어·충분한 길이·무 flag일 때 READY다.
경계	——	fallback·짧은 전사·reference 유사도 저하는 HOLD다.
외부 쓰기	——	STT 품질 Gate: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·quality_gate.json

STT 품질 Gate 파일 지도

입력·fixture

src/meeting_demo.py

구현 코드

tests/test_meeting_agent_workflow.py

검증·test

data/demo_meeting_transcript.txt

따라하기 notebook

output/day2-stt-lab/meeting_result.json

STT 품질 Gate 개념·코드

VAD

말소리 구간을 먼저 찾는 처리

```
src/meeting_demo.py
```

no-speech

해당 구간이 무음일 가능성

```
tests/test_meeting_agent_workflow.py
```

log probability

모델이 전사에 가진 상대 신뢰

```
data/demo_meeting_transcript.txt
```

Hotwords

제품명 등 인식 힌트

```
output/day2-stt-lab/meeting_result.json
```

STT 품질 Gate 선택 기준

구분	역할	이번 차시의 판단 기준
VAD	segment metric 수집 단계의 입력 기준	결정론 test로 먼저 확인
no-speech	반복 탐지 단계의 교체 경계	adapter 경계를 확인
log probability	언어 확인 단계의 운영 조건	비용·지연·권한을 확인
Hotwords	reference 비교 단계의 판단 기록	사람 판단과 운영 기록을 확인

STT 품질 Gate 대표 실패

실패

낮은 품질 segment 하나를 전체 성공으로 숨긴다.

사용자 영향

낮은 품질 segment 하나를 전체 성공으로 숨긴다.
상태에서는 quality_gate.json을 운영 판단에 사용할 수 없다.

STT 품질 Gate 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

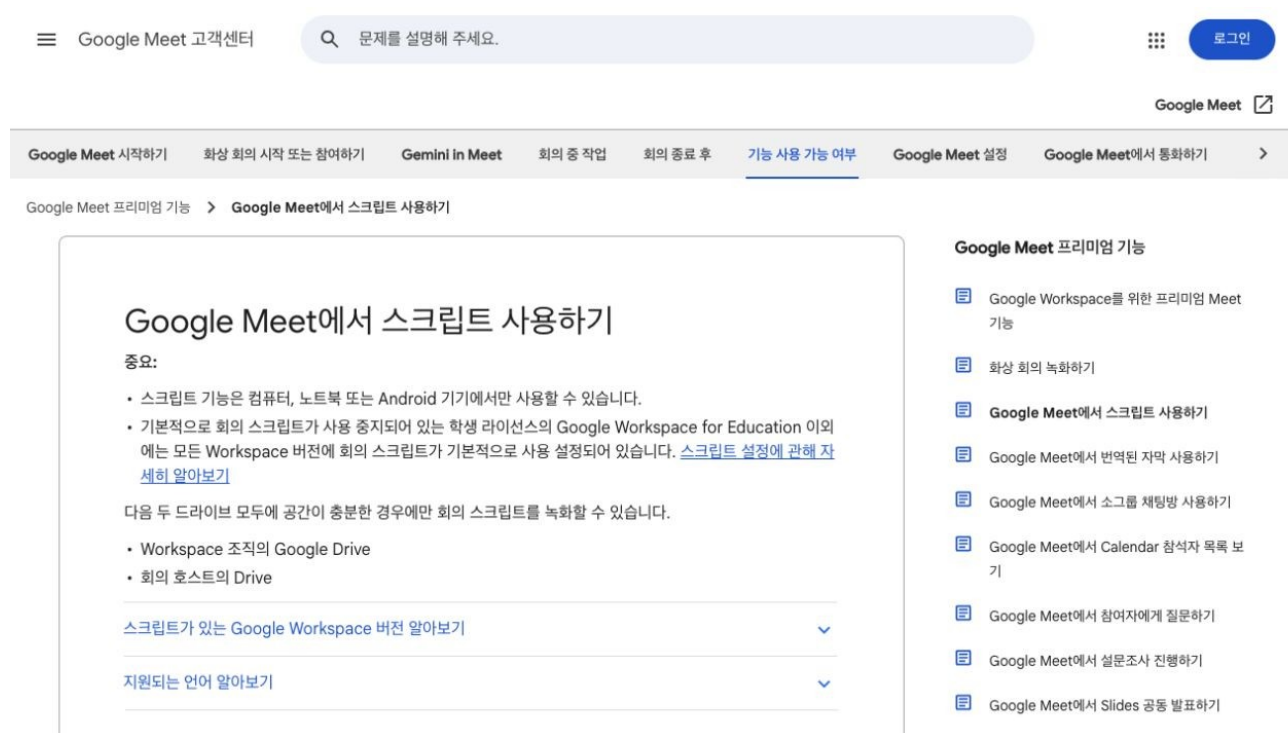
STT 품질 Gate 복구 경로

- 01 — 탐지: fallback·짧은 전사·reference 유사도 저하는 HOLD다.
- 02 — 중단: 낮은 품질 segment 하나를 전체 성공으로 숨긴다.
- 03 — 복구: flagged segment와 timestamp만 사람에게 보여주고 accept·edit·reject를 받는다.
- 04 — 증거: quality_gate.json과 test 결과를 함께 남긴다.

STT 품질 Gate Codex 검증 루프

tests/
test_meeting_agent_workflow.
py의 책임을 찾고
quality_gate.json 계약을
focused test로 고정합니다.

STT 품질 Gate
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



STT 품질 Gate Codex 작업 명세

목표

품질 threshold 변경이 기존 정상 음성을 HOLD로 만들지 회귀 test를 작성한다.

허용 경로

tests/test_meeting_agent_workflow.py
data/demo_meeting_transcript.txt

완료 조건

한국어·충분한 길이·무 flag일 때 READY다.
fallback·짧은 전사·reference 유사도 저하는 HOLD다.

금지

STT 품질 Gate: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

STT 품질 Gate 독립 리뷰

- | | | | |
|----|----------------------|----|--|
| 01 | Codex diff | —— | STT 품질 Gate 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 한국어·충분한 길이·무 flag일 때 READY다. |
| 03 | Claude review | —— | quality_gate.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | STT 품질 Gate의 두 LLM 의견과 test는 자동 승인이 아니다 |

STT 품질 Gate 정상 시연

- 01 — 열기: `src/meeting_demo.py`
- 02 — 구현 확인: `tests/test_meeting_agent_workflow.py`
- 03 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py`
- 04 — 성공 신호: 한국어·충분한 길이·무 flag일 때 READY다.

STT 품질 Gate 실행 명령

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py
```

실행 전 확인

STT 품질 Gate: repository root · Python 3.12 · .env 미출력

03_quality_gate.json 실행 결과

```
{
  "normal": {
    "decision": "READY",
    "flagged_segment_count": 0,
    "reasons": [],
    "human_decision_required": true,
    "detected_language": "ko",
    "language_probability": 0.99,
    "segment_count": 45,
    "text_length": 5141,
    "reference_similarity": 1,
    "minimum_reference_similarity": 0.8
  },
  "boundary": {
    "decision": "HOLD",
    "flagged_segment_count": 0,
    "reasons": [
      "NO_SPEECH_SEGMENTS",
      "STT_FALLBACK_USED",
      "TRANSCRIPT_TOO_SHORT"
    ],
    "human_decision_required": true,
    "detected_language": "ko",
    "language_probability": null,
    "segment_count": 0,
    ...
  }
}
```

STT 품질 Gate 실패·복구 시연

재현

fallback·짧은 전사·reference 유사도
저하는 HOLD다.

복구

flagged segment와 timestamp만
사람에게 보여주고 accept·edit·reject를
받는다.

확인: fallback·짧은 전사·reference 유사도 저하는 HOLD다.

STT 품질 Gate 소프트웨어 제작

열 파일

tests/test_meeting_agent_workflow.py
data/demo_meeting_transcript.txt

작업 범위

STT 품질 Gate 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

한국어·충분한 길이·무 flag일 때 READY다.
quality_gate.json 저장

STT 품질 Gate Notebook 셀

materials/day2/day2_service_lab.ipynb · 3차시

```
from src.meeting_demo import build_quality_gate

ready_gate = build_quality_gate(
    stt_segments, mode="local_stt",
    metadata={"language": "ko", "language_probability": 0.99},
    reference_similarity=1.0,
)
hold_gate = build_quality_gate([], mode="fixture", metadata={"language": "ko"})
quality_result = {"normal": ready_gate, "boundary": hold_gate}
assert ready_gate["decision"] == "READY"
assert hold_gate["decision"] == "HOLD"
save_json("03_quality_gate.json", quality_result)
quality_result
```

STT 품질 Gate Terminal 실행

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py
```

저장 위치

```
output/course-labs/day2/03_quality_gate.json
```

한국어·충분한 길이·무 flag일 때 READY다.
실패 시 quality_gate.json의 status·error_code를 먼저 확인

STT 품질 Gate 실패 증거

fallback·짧은 전사·reference 유사도 저하는 HOLD다.

```
result_file = "output/course-labs/day2/03_quality_gate.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

quality_gate.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

STT 품질 Gate 정상 Case

NORMAL

한국어·충분한 길이·무 flag일 때 READY다.

- 01 — src/meeting_demo.py 입력과 command가 기록됐는가
- 02 — quality_gate.json schema가 validate되는가
- 03 — STT 품질 Gate focused test가 실제로 통과했는가

STT 품질 Gate 경계 Case

BOUNDARY

fallback·짧은 전사·reference 유사도 저하는 HOLD다.

- 01 — STT 품질 Gate: raw traceback 대신 stable error_code가 남는가
- 02 — quality_gate.json: 외부 쓰기·자동 메일이 false인가
- 03 — fallback·짧은 전사·reference 유사도 저하는 HOLD다. 재현이 가능한가

STT 품질 Gate 현업 적용

회의록 생성 전에 고유명사와 금액 구간만 운영자가 spot check

- 01 — STT 품질 Gate은 합성·비식별 sample로 먼저 실행
- 02 — 회의록 생성 전에 고유명사와 금액 구간만 운영자가 spot check의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — quality_gate.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — quality_gate.json을 운영 검토 자료로 사용

STT 품질 Gate 포트폴리오 적용

STT 품질 Gate · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 낮은 품질 segment 하나를 전체 성공으로 숨긴다.의 사용자 영향을 한 문장으로 설명
- 02 — STT 품질 Gate 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 품질 threshold 변경이 기존 정상 음성을 HOLD로 만들지 회귀 test를 작성한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — quality_gate.json과 README 재현 절차를 제출

STT 품질 Gate 운영 점검

품질

무엇을 한국어·충분한 길이·무 flag일 때 READY다.로 정의하는가

복구

낮은 품질 segment 하나를 전체 성공으로 숨긴다. 발생 시 quality_gate.json에서 원인 상태를 확인

안전

STT 품질 Gate의 사람 승인과 external_write=false 위치

관측

quality_gate.json에서 어떤 status·error_code를 보는가

quality_gate.json 실행 증거

- 01 — 설명: STT 품질 Gate이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py`
- 03 — 증거: quality_gate.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 회의록 Schema와 Pydantic 계약

12:00-12:50 (쉬는 시간 13:40-14:00)

회의록 Schema와 Pydantic 계약

Schema · Pydantic · meeting_schema.json

진행

회의록 Schema와 Pydantic 계약
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/langchain_lab.py
src/course_services/
meeting_service.py

확인할 결과

meeting_schema.json

4차시 입력과 결과

입력	————	quality_gate.json
이번 차시	————	회의록 Schema와 Pydantic 계약
결과	————	meeting_schema.json
다음 연결	————	발화 단위 Chunking

회의록 Schema와 Pydantic 계약 필요성

자유로운 문장은 읽기 좋지만 downstream 자동화가 실패해도 원인을 찾기 어렵다.

필수 필드 선언에서 meeting_schema.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: meeting_schema.json

회의록 Schema와 Pydantic 계약 학습 결과

- 01 — summary·decision·action item·evidence를 Pydantic 계약으로 만든다.
- 02 — 원문에 없는 담당자와 날짜를 빈칸 대신 추측한다. 왜 위험한지 설명한다.
- 03 — meeting_schema.json에서 정상과 실패 상태를 직접 확인한다.

회의록 Schema와 Pydantic 계약 용어

용어	이 수업에서 쓰는 뜻
Schema	결과 필드와 형식의 약속
Pydantic	Python 데이터 검증 도구
Structured Output	정해진 구조로 받는 모델 결과
null	원문에 값이 없음을 정직하게 표시

meeting_schema.json 입출력

INPUT

src/langchain_lab.py

OUTPUT

meeting_schema.json

PROCESS

필수 필드 선언 → type·범위 검사 → 추가 필드 차단

VERIFY

세 개 Action Item이 evidence_ids와 함께 validate된다.
automatic_email=true 또는 evidence 누락은 validation error다.

회의록 Schema와 Pydantic 계약 실행 흐름

01

필수 필드 선언

02

type·범위 검사

03

추가 필드 차단

04

evidence 확인

05

validation error
반환

마지막 판단은 meeting_schema.json에서 사람이 확인합니다.

meeting_schema.json 정상·실패 계약

정상	세 개 Action Item이 evidence_ids와 함께 validate된다.
경계	automatic_email=true 또는 evidence 누락은 validation error다.
외부 쓰기	회의록 Schema와 Pydantic 계약: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	status·error_code·입력 식별자·meeting_schema.json

회의록 Schema와 Pydantic 계약 파일 지도

입력·fixture

src/langchain_lab.py

구현 코드

src/course_services/meeting_service.py

검증·test

tests/test_langchain_langgraph_lab.py

따라하기 notebook

materials/day2/day2_service_lab.ipynb

회의록 Schema와 Pydantic 계약 개념·코드

Schema

결과 필드와 형식의 약속

```
src/langchain_lab.py
```

Structured Output

정해진 구조로 받는 모델 결과

```
tests/test_langchain_langgraph_lab.py
```

Pydantic

Python 데이터 검증 도구

```
src/course_services/meeting_service.py
```

null

원문에 값이 없음을 정직하게 표시

```
materials/day2/day2_service_lab.ipynb
```

회의록 Schema와 Pydantic 계약 선택 기준

구분	역할	이번 차시의 판단 기준
Schema	필수 필드 선언 단계의 입력 기준	결정론 test로 먼저 확인
Pydantic	type·범위 검사 단계의 교체 경계	adapter 경계를 확인
Structured Output	추가 필드 차단 단계의 운영 조건	비용·지연·권한을 확인
null	evidence 확인 단계의 판단 기록	사람 판단과 운영 기록을 확인

회의록 Schema와 Pydantic 계약 대표 실패

실패

원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.

사용자 영향

원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.
상태에서는 meeting_schema.json을 운영
판단에 사용할 수 없다.

회의록 Schema와 Pydantic 계약 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

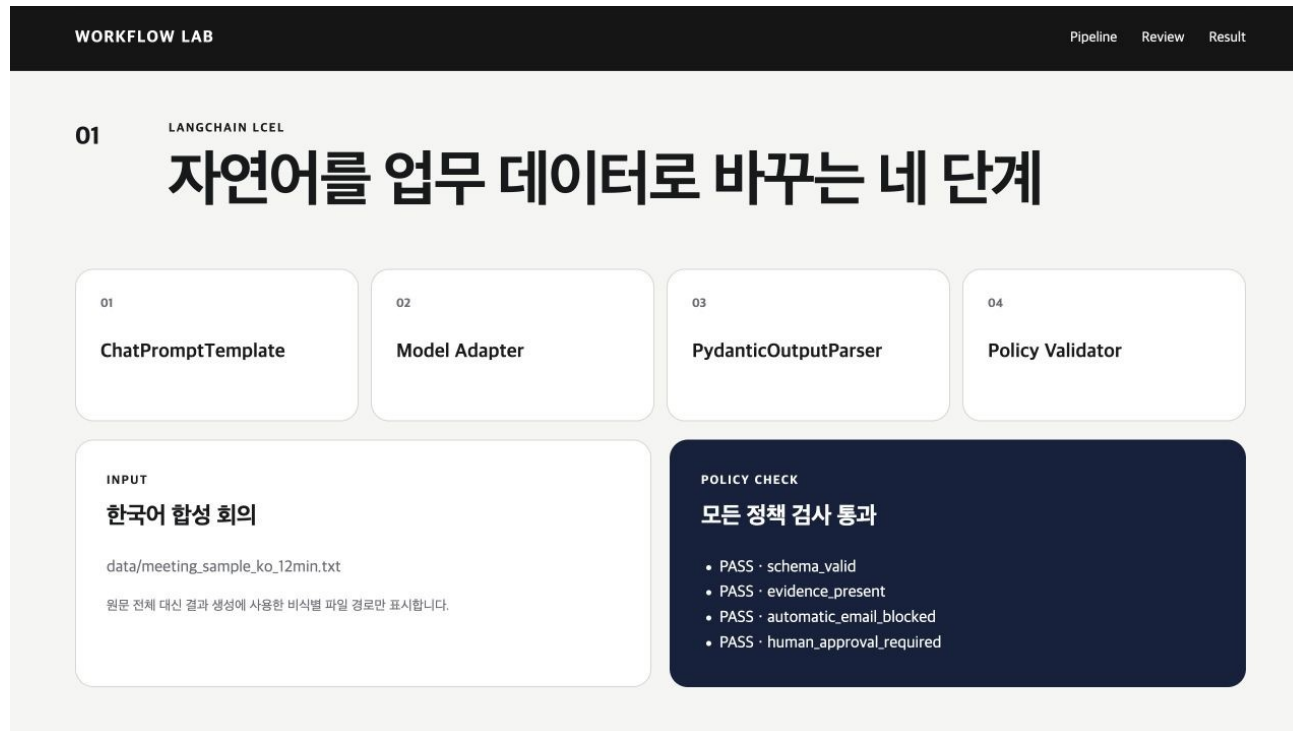
회의록 Schema와 Pydantic 계약 복구 경로

- 01 — 탐지: `automatic_email=true` 또는 `evidence` 누락은 `validation error`다.
- 02 — 중단: 원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.
- 03 — 복구: 미상값은 `null`로 두고 `evidence`가 없는 Action Item은 HOLD한다.
- 04 — 증거: `meeting_schema.json`과 test 결과를 함께 남긴다.

회의록 Schema와 Pydantic 계약 Codex 검증 루프

src/course_services/
meeting_service.py의 책임을
찾고 meeting_schema.json
계약을 focused test로 고정합니다.

회의록 Schema와 Pydantic 계약
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



회의록 Schema와 Pydantic 계약 Codex 작업 명세

목표

회의 schema에 필드를 추가하고 extra=forbid·누락 field test를 함께 만든다.

허용 경로

src/course_services/meeting_service.py
tests/test_langchain_langgraph_lab.py

완료 조건

세 개 Action Item이 evidence_ids와 함께 validate된다.
automatic_email=true 또는 evidence 누락은 validation error다.

금지

회의록 Schema와 Pydantic 계약: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

회의록 Schema와 Pydantic 계약 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | 회의록 Schema와 Pydantic 계약 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 세 개 Action Item이 evidence_ids와 함께 validate된다. |
| 03 | Claude review | —— | meeting_schema.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 회의록 Schema와 Pydantic 계약의 두 LLM 의견과 test는 자동 승인이 아니다 |

회의록 Schema와 Pydantic 계약 정상 시연

- 01 — 열기: `src/langchain_lab.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m src.langchain_lab --provider fixture`
- 04 — 성공 신호: 세 개 Action Item이 `evidence_ids`와 함께 validate된다.

회의록 Schema와 Pydantic 계약 실행 명령

```
$ python -m src.langchain_lab --provider fixture
```

실행 전 확인

회의록 Schema와 Pydantic 계약: repository root · Python 3.12 · .env 미출력

04_meeting_schema.json 실행 결과

```
{
  "normal": {
    "title": "고객 문의 자동화 PoC 범위 회의",
    "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
    "decisions": [
      "배송 지연과 반품 절차 안내만 자동화한다.",
      "개인정보·근거 부족·낮은 신뢰도는 상담원 검토 큐로 보낸다."
    ],
    "action_items": [
      {
        "task": "공개 FAQ 30건 비식별 샘플 정리",
        "owner": "서연",
        "due_date": "2026-08-27",
        "evidence_ids": [
          "s12",
          "s18"
        ]
      },
      {
        "task": "응답 JSON Schema와 실패 테스트 작성",
        "owner": "준호",
        "due_date": "2026-08-28",
        "evidence_ids": [
          "s27",
          "s31"
        ]
      }
    ]
  },
  ...
}
```

회의록 Schema와 Pydantic 계약 실패·복구 시연

재현

automatic_email=true 또는 evidence 누락은 validation error다.

복구

미상값은 null로 두고 evidence가 없는 Action Item은 HOLD한다.

확인: automatic_email=true 또는 evidence 누락은 validation error다.

회의록 Schema와 Pydantic 계약 소프트웨어 제작

열 파일	<code>src/course_services/meeting_service.py</code> <code>tests/test_langchain_langgraph_lab.py</code>
작업 범위	회의록 Schema와 Pydantic 계약 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	세 개 Action Item이 evidence_ids와 함께 validate된다. meeting_schema.json 저장

회의록 Schema와 Pydantic 계약 Notebook 셀

materials/day2/day2_service_lab.ipynb · 4차시

```
from pydantic import ValidationError
from src.langchain_lab import ActionItem, MeetingBrief, fixture_payload

meeting_brief = MeetingBrief.model_validate(fixture_payload())
try:
    ActionItem(task="근거 없는 할 일", owner="미정", due_date="2026-08-30", evidence_ids=[])
    schema_boundary = {"status": "UNEXPECTED_SUCCESS"}
except ValidationError as exc:
    schema_boundary = {"status": "EXPECTED_FAILURE", "error_code": exc.errors()[0]["type"]}
schema_result = {"normal": meeting_brief.model_dump(mode="json"), "boundary": schema_boundary}
save_json("04_meeting_schema.json", schema_result)
{"title": meeting_brief.title, "boundary": schema_boundary}
```


회의록 Schema와 Pydantic 계약 Terminal 실행

```
$ python -m src.langchain_lab --provider fixture
```

저장 위치

```
output/course-labs/day2/04_meeting_schema.json
```

세 개 Action Item이 evidence_ids와 함께 validate된다.
실패 시 meeting_schema.json의 status·error_code를 먼저 확인

회의록 Schema와 Pydantic 계약 실패 증거

`automatic_email=true` 또는 `evidence` 누락은 validation error다.

```
result_file = "output/course-labs/day2/04_meeting_schema.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

`meeting_schema.json`이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

회의록 Schema와 Pydantic 계약 정상 Case

NORMAL

세 개 Action Item이 evidence_ids와 함께 validate된다.

- 01 — src/langchain_lab.py 입력과 command가 기록됐는가
- 02 — meeting_schema.json schema가 validate되는가
- 03 — 회의록 Schema와 Pydantic 계약 focused test가 실제로 통과했는가

회의록 Schema와 Pydantic 계약 경계 Case

BOUNDARY

automatic_email=true 또는 evidence 누락은 validation error다.

- 01 — 회의록 Schema와 Pydantic 계약: raw traceback 대신 stable error_code가 남는가
- 02 — meeting_schema.json: 외부 쓰기·자동 메일이 false인가
- 03 — automatic_email=true 또는 evidence 누락은 validation error다. 재현이 가능한가

회의록 Schema와 Pydantic 계약 현업 적용

회의 요약을 Jira·Notion·메일로 보내기 전 공통 계약

- 01 — 회의록 Schema와 Pydantic 계약은 합성·비식별 sample로 먼저 실행
- 02 — 회의 요약을 Jira·Notion·메일로 보내기 전 공통 계약의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — meeting_schema.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_schema.json을 운영 검토 자료로 사용

회의록 Schema와 Pydantic 계약 포트폴리오 적용

회의록 Schema와 Pydantic 계약 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 원문에 없는 담당자와 날짜를 빈칸 대신 추측한다.의 사용자 영향을 한 문장으로 설명
- 02 — 회의록 Schema와 Pydantic 계약 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 회의 schema에 필드를 추가하고 extra=forbid·누락 field test를 함께 만든다.의 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_schema.json과 README 재현 절차를 제출

회의록 Schema와 Pydantic 계약 운영 점검

품질

무엇을 세 개 Action Item이 evidence_ids와 함께 validate된다.로 정의하는가

복구

원문에 없는 담당자와 날짜를 빈칸 대신 추측한다. 발생 시 meeting_schema.json에서 원인 상태를 확인

안전

회의록 Schema와 Pydantic 계약의 사람 승인과 external_write=false 위치

관측

meeting_schema.json에서 어떤 status·error_code를 보는가

meeting_schema.json 실행 증거

- 01 — 설명: 회의록 Schema와 Pydantic 계약이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langchain_lab --provider fixture`
- 03 — 증거: meeting_schema.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 발화 단위 Chunking

12:50-13:40 (쉬는 시간 13:40-14:00)

발화 단위 Chunking

Chunk · Overlap · meeting_chunks.json

진행

발화 단위 Chunking
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

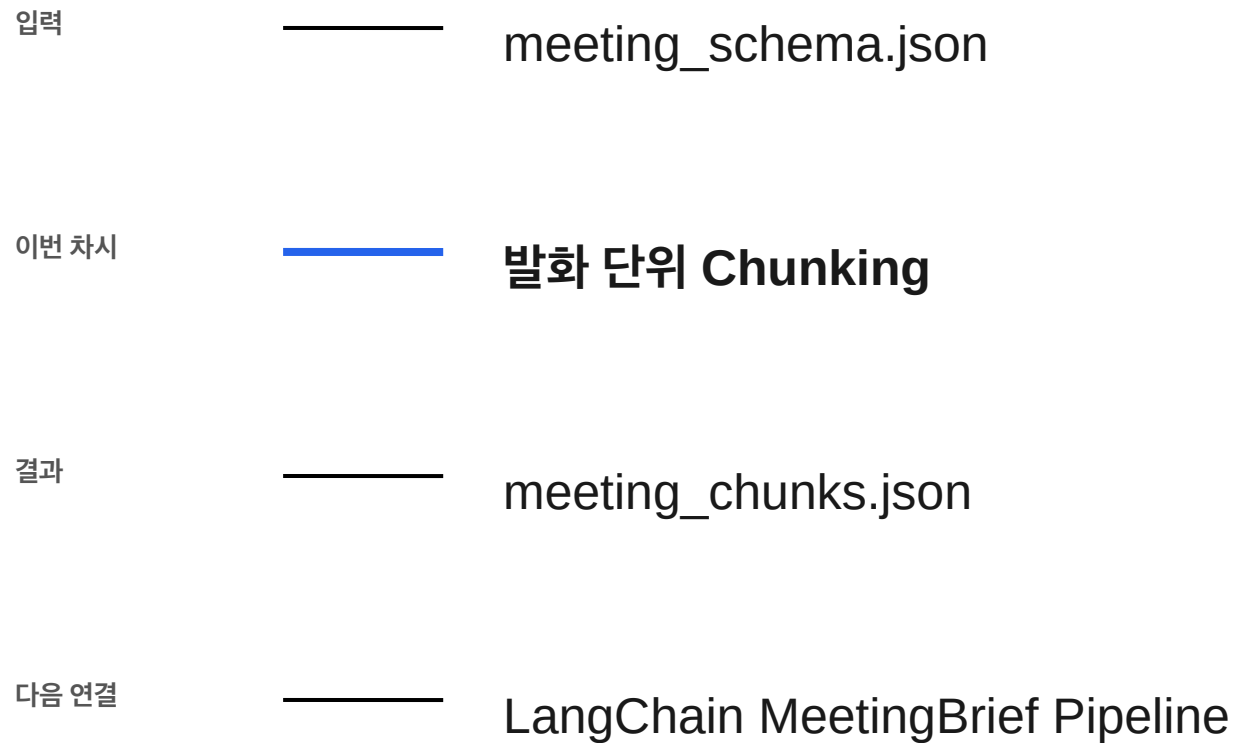
사용 파일

src/course_services/
meeting_service.py
tests/test_course_services.py

확인할 결과

meeting_chunks.json

5차시 입력과 결과



발화 단위 Chunking 필요성

글자 수만 자르면 담당자·결정·근거가 서로 다른 chunk로 갈라진다.

segment 정렬에서 meeting_chunks.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: meeting_chunks.json

발화 단위 Chunking 학습 결과

- 01 — timestamp와 evidence ID를 잃지 않는 chunk를 만든다.
- 02 — overlap 때문에 같은 Action Item이 두 번 생성된다. 왜 위험한지 설명한다.
- 03 — meeting_chunks.json에서 정상과 실패 상태를 직접 확인한다.

발화 단위 Chunking 용어

용어	이 수업에서 쓰는 뜻
Chunk	모델에 넣을 작은 입력 묶음
Overlap	경계 문맥을 이어 주는 중복 구간
Map	chunk별 초안을 만드는 단계
Reduce	여러 초안을 하나로 합치는 단계

meeting_chunks.json 입출력

INPUT

src/course_services/meeting_service.py

OUTPUT

meeting_chunks.json

PROCESS

segment 정렬 → 길이 누적 → 경계에서 close

VERIFY

모든 segment ID가 적어도 한 chunk에 존재한다.
max_chars<40과 음수 overlap은 명시 오류다.

발화 단위 Chunking 실행 흐름

01

segment 정렬

02

길이 누적

03

경계에서 close

04

한 segment overlap

05

chunk ID 저장

마지막 판단은 `meeting_chunks.json`에서 사람이 확인합니다.

meeting_chunks.json 정상·실패 계약

정상	——	모든 segment ID가 적어도 한 chunk에 존재한다.
경계	——	max_chars<40과 음수 overlap은 명시 오류다.
외부 쓰기	——	발화 단위 Chunking: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·meeting_chunks.json

발화 단위 Chunking 파일 지도

입력·fixture

`src/course_services/meeting_service.py`

구현 코드

`tests/test_course_services.py`

검증·test

`data/meeting_sample_ko_12min.txt`

따라하기 notebook

`materials/day2/day2_service_lab.ipynb`

발화 단위 Chunking 개념·코드

Chunk

모델에 넣을 작은 입력 묶음

```
src/course_services/meeting_service.py
```

Map

chunk별 초안을 만드는 단계

```
data/meeting_sample_ko_12min.txt
```

Overlap

경계 문맥을 이어 주는 중복 구간

```
tests/test_course_services.py
```

Reduce

여러 초안을 하나로 합치는 단계

```
materials/day2/day2_service_lab.ipynb
```

발화 단위 Chunking 선택 기준

구분	역할	이번 차시의 판단 기준
Chunk	segment 정렬 단계의 입력 기준	결정론 test로 먼저 확인
Overlap	길이 누적 단계의 교체 경계	adapter 경계를 확인
Map	경계에서 close 단계의 운영 조건	비용·지연·권한을 확인
Reduce	한 segment overlap 단계의 판단 기록	사람 판단과 운영 기록을 확인

발화 단위 Chunking 대표 실패

실패

overlap 때문에 같은 Action Item이 두 번 생성된다.

사용자 영향

overlap 때문에 같은 Action Item이 두 번 생성된다. 상태에서는 meeting_chunks.json을 운영 판단에 사용할 수 없다.

발화 단위 Chunking 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

발화 단위 Chunking 복구 경로

- 01 — 탐지: max_chars<40과 음수 overlap은 명시 오류다.
- 02 — 중단: overlap 때문에 같은 Action Item이 두 번 생성된다.
- 03 — 복구: evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.
- 04 — 증거: meeting_chunks.json과 test 결과를 함께 남긴다.

발화 단위 Chunking Codex 검증 루프

`tests/test_course_services.py`
의 책임을 찾고
`meeting_chunks.json` 계약을
`focused test`로 고정합니다.

발화 단위 Chunking
저장소 이해 → 허용 범위 변경
→ `focused test` → `diff` 리뷰
→ 사람 merge



Loading JupyterLite...

발화 단위 Chunking Codex 작업 명세

목표

segment 경계를 보존하는 chunk 함수와 작은 max_chars failure test를 작성한다.

허용 경로

tests/test_course_services.py
data/meeting_sample_ko_12min.txt

완료 조건

모든 segment ID가 적어도 한 chunk에 존재한다.
max_chars<40과 음수 overlap은 명시 오류다.

금지

발화 단위 Chunking: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

발화 단위 Chunking 독립 리뷰

- | | | | |
|----|----------------------|------|--|
| 01 | Codex diff | ———— | 발화 단위 Chunking 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | 모든 segment ID가 적어도 한 chunk에 존재한다. |
| 03 | Claude review | ———— | meeting_chunks.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | 발화 단위 Chunking의 두 LLM 의견과 test는 자동 승인이 아니다 |

발화 단위 Chunking 정상 시연

- 01 — 열기: `src/course_services/meeting_service.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k meeting_chunks`
- 04 — 성공 신호: 모든 segment ID가 적어도 한 chunk에 존재한다.

발화 단위 Chunking 실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k meeting_chunks
```

실행 전 확인

발화 단위 Chunking: repository root · Python 3.12 · .env 미출력

05_meeting_chunks.json 실행 결과

```
{
  "chunks": [
    {
      "id": "c01",
      "segment_ids": [
        "s01",
        "s02",
        "s03",
        "s04",
        "s05",
        "s06",
        "s07"
      ],
      "text": "\n모두 들어오셨죠? 오늘 회의 목적은 고객 문의 자동화 PoC의 첫 범위를 확정하는 것입니다. 모델을 고르는 회의가 아니라, 어떤 문의를 어디까지 자동화하고 어디서 사람이 확인할지 정하는 회의로 진행하겠습니다.\n\n운영 쪽 현황부터 말씀드릴게요. 최근 4주 동안 합성 기준으로 문의가 1,240건 정도였고, 배송 지연이 31퍼센트, 반품 절차가 24퍼센트, 환불과 보상 관련 문의가 18퍼센트였습니다. 나머지는 계정, 쿠폰, 상품 정보처럼 유형이 조금씩 달랐습니다.\n\n그중에서 API 없이도 답할 수 있는 범위를 먼저 골라야 할 것 같습니다. 배송 지연은 주문 상태 조회가 필요할 수 있고, 반품 절차는 정책 문서만으로도 초안 생성이 가능하니까 같은 자동화로 묶으면 위험도가 다릅니다.\n\n맞습니다. 그래서 입력을 두 종류로 나누면 좋겠습니다. 첫 번째는 공개 FAQ와 합성 문의만 사용하는 교육용 경로이고, 두 번째는 주문 상태 같은 내부 도구를 연결하는 확장 경로입니다. 오늘은 첫 번째 경로만 구현하는 게 안전합니다.\n\n그러면 오늘 결정할 PoC 사용자는 상담원 한 명으로 두죠. 고객에게 바로 답하는 Agent가 아니라, 상담원이 검토할 답변 초안과 근거 문장을 만드는 Copilot으로 정의하겠습니다.\n\n동의합니다. 실제 상담에서는 답변 자체보다 근거를 찾는 시간이 오래 걸립니다. 문의 한 건당 평균 처리 시간이 6분 40초 정도라고 가정하면, FAQ를 찾고 초안을 만드는 시간을 30초 안으로 줄이는 것이 1차 가치가 될 수 있습니다.\n\n외부 발송은 이번 범위에서 빼는 게 좋겠습니다. send_email이나 상담 시스템 게시 도구는 등록하지 않고, 로컬 Markdown이나 JSON 초안까지만 저장하도록 하겠습니다."
    },
    {
      "id": "c02",
      "segment_ids": [
        "s07",
        "s08",
        "s09",
        "s10",
        "s11",
        "s12",
        "s13",
        ...
      ]
    }
  ]
}
```

발화 단위 Chunking 실패·복구 시연

재현

max_chars<40과 음수 overlap은 명시 오류다.

복구

evidence ID 기반 deduplication과 최종 사람 검토를 적용한다.

확인: max_chars<40과 음수 overlap은 명시 오류다.

발화 단위 Chunking 소프트웨어 제작

열 파일

tests/test_course_services.py
data/meeting_sample_ko_12min.txt

작업 범위

발화 단위 Chunking 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

모든 segment ID가 적어도 한 chunk에 존재한다.
meeting_chunks.json 저장

발화 단위 Chunking Notebook 셀

materials/day2/day2_service_lab.ipynb · 5차시

```
from src.course_services.meeting_service import chunk_transcript_segments

chunks = chunk_transcript_segments(stt_segments, max_chars=900, overlap_segments=1)
try:
    chunk_transcript_segments(stt_segments, max_chars=20)
    chunk_boundary = {"status": "UNEXPECTED_SUCCESS"}
except ValueError as exc:
    chunk_boundary = {"status": "EXPECTED_FAILURE", "error_code": str(exc)}
chunk_result = {"chunks": chunks, "boundary": chunk_boundary}
save_json("05_meeting_chunks.json", chunk_result)
{"chunk_count": len(chunks), "boundary": chunk_boundary}
```

발화 단위 Chunking Terminal 실행

```
$ python -m pytest -q tests/test_course_services.py -k meeting_chunks
```

저장 위치

```
output/course-labs/day2/05_meeting_chunks.json
```

모든 segment ID가 적어도 한 chunk에 존재한다.
실패 시 meeting_chunks.json의 status·error_code를 먼저 확인

발화 단위 Chunking 실패 증거

max_chars<40과 음수 overlap은 명시 오류다.

```
result_file = "output/course-labs/day2/05_meeting_chunks.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

meeting_chunks.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

발화 단위 Chunking 정상 Case

NORMAL

모든 segment ID가 적어도 한 chunk에 존재한다.

- 01 — src/course_services/meeting_service.py 입력과 command가 기록됐는가
- 02 — meeting_chunks.json schema가 validate되는가
- 03 — 발화 단위 Chunking focused test가 실제로 통과했는가

발화 단위 Chunking 경계 Case

BOUNDARY

max_chars<40과 음수 overlap은 명시 오류다.

- 01 — 발화 단위 Chunking: raw traceback 대신 stable error_code가 남는가
- 02 — meeting_chunks.json: 외부 쓰기·자동 메일이 false인가
- 03 — max_chars<40과 음수 overlap은 명시 오류다. 재현이 가능한가

발화 단위 Chunking 현업 적용

10분 이상 상품 운영 회의를 결정·담당자·기한으로 축약

- 01 — 발화 단위 Chunking은 합성·비식별 sample로 먼저 실행
- 02 — 10분 이상 상품 운영 회의를 결정·담당자·기한으로 축약의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — meeting_chunks.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_chunks.json을 운영 검토 자료로 사용

발화 단위 Chunking 포트폴리오 적용

발화 단위 Chunking · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — overlap 때문에 같은 Action Item이 두 번 생성된다.의 사용자 영향을 한 문장으로 설명
- 02 — 발화 단위 Chunking 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — segment 경계를 보존하는 chunk 함수와 작은 max_chars failure test를 작성한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_chunks.json과 README 재현 절차를 제출

발화 단위 Chunking 운영 점검

품질

무엇을 모든 segment ID가 적어도 한 chunk에 존재한다.
로 정의하는가

복구

overlap 때문에 같은 Action Item이 두 번 생성된다. 발생
시 meeting_chunks.json에서 원인 상태를 확인

안전

발화 단위 Chunking의 사람 승인과
external_write=false 위치

관측

meeting_chunks.json에서 어떤 status_error_code
를 보는가

meeting_chunks.json 실행 증거

- 01 — 설명: 발화 단위 Chunking이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k meeting_chunks`
- 03 — 증거: meeting_chunks.json과 정상·실패 test 결과가 남아 있다.

다음 차시: LangChain MeetingBrief Pipeline

DAY 2 · 6차시

15:00-15:50 (쉬는 시간 17:30-18:00)

LangChain MeetingBrief Pipeline

Runnable · LCEL · meeting_brief.json

진행

LangChain MeetingBrief Pipeline
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

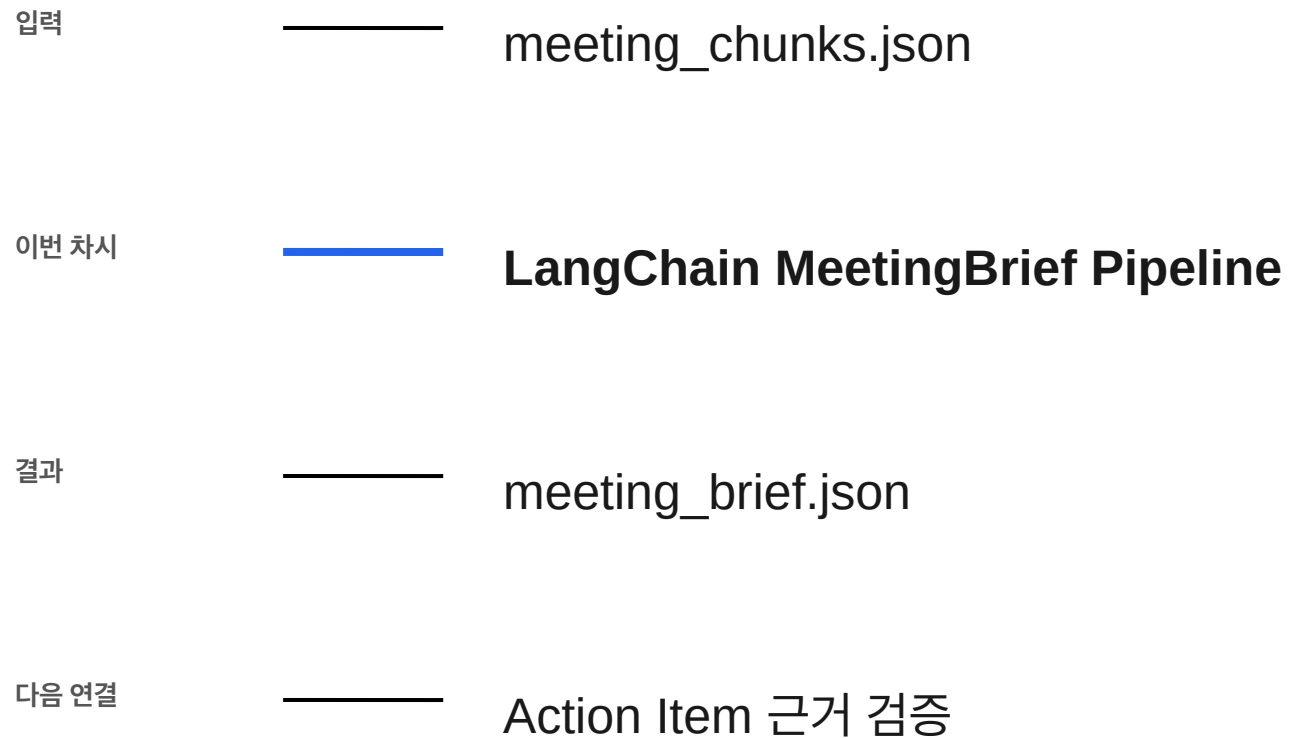
사용 파일

src/langchain_lab.py
src/openai_provider.py

확인할 결과

meeting_brief.json

6차시 입력과 결과



LangChain MeetingBrief Pipeline 필요성

provider별 코드를 업무 로직에 섞으면 모델 변경이 곧 전체 재개발이 된다.

prompt 구성에서 meeting_brief.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: meeting_brief.json

LangChain MeetingBrief Pipeline 학습 결과

- 01 — fixture·Ollama·OpenAI가 같은 MeetingBrief를 반환하게 한다.
- 02 — OpenAI 오류가 조용히 fixture 성공처럼 보인다. 왜 위험한지 설명한다.
- 03 — meeting_brief.json에서 정상과 실패 상태를 직접 확인한다.

LangChain MeetingBrief Pipeline 용어

용어	이 수업에서 쓰는 뜻
Runnable	같은 invoke 계약으로 실행되는 구성요소
LCEL	Runnable을 연결하는 표현 방식
Adapter	provider 차이를 흡수하는 층
Fallback	실패 시 정해진 대체 경로

meeting_brief.json 입출력

INPUT

src/langchain_lab.py

OUTPUT

meeting_brief.json

PROCESS

prompt 구성 → provider 선택 → structured generation

VERIFY

Qwen과 fixture가 같은 필수 field를 반환한다.
provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

LangChain MeetingBrief Pipeline 실행 흐름

01

prompt 구성

02

provider 선택

03

structured
generation

04

Pydantic parse

05

policy validate

마지막 판단은 meeting_brief.json에서 사람이 확인합니다.

meeting_brief.json 정상·실패 계약

정상	——	Qwen과 fixture가 같은 필수 field를 반환한다.
경계	——	provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
외부 쓰기	——	LangChain MeetingBrief Pipeline: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·meeting_brief.json

LangChain MeetingBrief Pipeline 파일 지도

입력·fixture

src/langchain_lab.py

구현 코드

src/openai_provider.py

검증·test

requirements-local-llm-optional.txt

따라하기 notebook

tests/test_langchain_langgraph_lab.py

LangChain MeetingBrief Pipeline 개념·코드

Runnable

같은 invoke 계약으로 실행되는 구성요소

```
src/langchain_lab.py
```

LCEL

Runnable을 연결하는 표현 방식

```
src/openai_provider.py
```

Adapter

provider 차이를 흡수하는 층

```
requirements-local-llm-optional.txt
```

Fallback

실패 시 정해진 대체 경로

```
tests/test_langchain_langgraph_lab.py
```


LangChain MeetingBrief Pipeline 선택 기준

구분	역할	이번 차시의 판단 기준
Runnable	prompt 구성 단계의 입력 기준	결정론 test로 먼저 확인
LCEL	provider 선택 단계의 교체 경계	adapter 경계를 확인
Adapter	structured generation 단계의 운영 조건	비용·지연·권한을 확인
Fallback	Pydantic parse 단계의 판단 기록	사람 판단과 운영 기록을 확인

LangChain MeetingBrief Pipeline 대표 실패

실패

OpenAI 오류가 조용히 fixture 성공처럼 보인다.

사용자 영향

OpenAI 오류가 조용히 fixture 성공처럼 보인다. 상태에서는 meeting_brief.json을 운영 판단에 사용할 수 없다.

LangChain MeetingBrief Pipeline 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

LangChain MeetingBrief Pipeline 복구 경로

- 01 — 탐지: provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.
- 02 — 중단: OpenAI 오류가 조용히 fixture 성공처럼 보인다.
- 03 — 복구: provider_requested·provider_used·fallback_reason을 결과에 함께 남긴다.
- 04 — 증거: meeting_brief.json과 test 결과를 함께 남긴다.

LangChain MeetingBrief Pipeline Codex 검증 루프

src/openai_provider.py의 책임을
찾고 meeting_brief.json 계약을
focused test로 고정합니다.

LangChain MeetingBrief Pipeline

저장소 이해 → 허용 범위 변경

→ focused test → diff 리뷰

→ 사람 merge

6차시 · LangChain LCEL Provider 교체

같은 transcript와 Pydantic schema를 유지하고 provider만 바꿉니다.

Prompt → fixture/Ollama → Pydantic Parser → Policy Validator

Ollama용 선택 패키지는 다음과 같이 설치합니다.

python -m pip install -r requirements-local-llm-optional.txt

```
In [10]: from src.langchain_lab import run_langchain_lab

transcript = (ROOT / "data/meeting_sample_ko_12min.txt").read_text(encoding="utf-8")
fixture_chain = run_langchain_lab(transcript, provider="fixture")
print(json.dumps({
    "provider_used": fixture_chain["provider_used"],
    "pipeline": fixture_chain["pipeline"],
    "checks": fixture_chain["checks"],
}, ensure_ascii=False, indent=2))

{
  "provider_used": "fixture",
  "pipeline": [
    "ChatPromptTemplate",
    "Model Adapter",
    "PydanticOutputParser",
    "Policy Validator"
  ],
  "checks": {
    "schema_valid": true,
    "evidence_present": true,
    "automatic_email_blocked": true,
    "human_approval_required": true
  }
}

In [11]: ollama_chain = run_langchain_lab(
    transcript,
    provider="ollama",
    model=MODEL,
    allow_fallback=True,
```

LangChain MeetingBrief Pipeline Codex 작업 명세

목표

새 provider adapter가 기존 MeetingBrief contract와 fallback test를 통과하게 한다.

허용 경로

src/openai_provider.py
requirements-local-llm-optional.txt

완료 조건

Qwen과 fixture가 같은 필수 field를 반환한다.
provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

금지

LangChain MeetingBrief Pipeline: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

LangChain MeetingBrief Pipeline 독립 리뷰

- | | | | |
|----|----------------------|------|---|
| 01 | Codex diff | ———— | LangChain MeetingBrief Pipeline 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | ———— | Qwen과 fixture가 같은 필수 field를 반환한다. |
| 03 | Claude review | ———— | meeting_brief.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | ———— | LangChain MeetingBrief Pipeline의 두 LLM 의견과 test는 자동 승인이 아니다 |

LangChain MeetingBrief Pipeline 정상 시연

- 01 — 열기: `src/langchain_lab.py`
- 02 — 구현 확인: `src/openai_provider.py`
- 03 — 실행: `python -m src.langchain_lab --provider ollama --model qwen3:4b`
- 04 — 성공 신호: Qwen과 fixture가 같은 필수 field를 반환한다.

LangChain MeetingBrief Pipeline 실행 명령

```
$ python -m src.langchain_lab --provider ollama --model qwen3:4b
```

실행 전 확인

LangChain MeetingBrief Pipeline: repository root · Python 3.12 · .env 미출력

06_meeting_brief.json 실행 결과

```
{
  "status": "SUCCESS",
  "framework": "LangChain LCEL",
  "provider_requested": "fixture",
  "provider_used": "fixture",
  "fallback_reason": null,
  "pipeline": [
    "ChatPromptTemplate",
    "Model Adapter",
    "PydanticOutputParser",
    "Policy Validator"
  ],
  "result": {
    "title": "고객 문의 자동화 PoC 범위 회의",
    "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
    "decisions": [
      "배송 지연과 반품 절차 안내만 자동화한다.",
      "개인정보·근거 부족·낮은 신뢰도는 상담원 검토 큐로 보낸다."
    ],
    "action_items": [
      {
        "task": "공개 FAQ 30건 비식별 샘플 정리",
        "owner": "서연",
        "due_date": "2026-08-27",
        "evidence_ids": [
          ...
        ]
      }
    ]
  }
}
```

LangChain MeetingBrief Pipeline 실패·복구 시연

재현

**provider 실패 시 허용 여부에 따라
fallback 또는 구조화 오류가 난다.**

복구

**provider_requested·provider_used·
fallback_reason을 결과에 함께 남긴다.**

확인: provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

LangChain MeetingBrief Pipeline 소프트웨어 제작

열 파일

src/openai_provider.py
requirements-local-llm-optional.txt

작업 범위

LangChain MeetingBrief Pipeline 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

Qwen과 fixture가 같은 필수 field를 반환한다.
meeting_brief.json 저장

LangChain MeetingBrief Pipeline Notebook 셀

materials/day2/day2_service_lab.ipynb · 6차시

```
from src.langchain_lab import run_langchain_lab

RUN_OLLAMA_LIVE = False
provider = "ollama" if RUN_OLLAMA_LIVE else "fixture"
chain_result = run_langchain_lab(transcript_text, provider=provider, allow_fallback=True)
assert chain_result["checks"]["schema_valid"] is True
assert chain_result["result"]["automatic_email"] is False
save_json("06_meeting_brief.json", chain_result)
{key: chain_result[key] for key in ("provider_requested", "provider_used", "fallback_reason")}
```

LangChain MeetingBrief Pipeline Terminal 실행

```
$ python -m src.langchain_lab --provider ollama --model qwen3:4b
```

저장 위치

output/course-labs/day2/06_meeting_brief.json

Qwen과 fixture가 같은 필수 field를 반환한다.
실패 시 meeting_brief.json의 status·error_code를 먼저 확인

LangChain MeetingBrief Pipeline 실패 증거

provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

```
result_file = "output/course-labs/day2/06_meeting_brief.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

meeting_brief.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

LangChain MeetingBrief Pipeline 정상 Case

NORMAL

Qwen과 fixture가 같은 필수 field를 반환한다.

- 01 — src/langchain_lab.py 입력과 command가 기록됐는가
- 02 — meeting_brief.json schema가 validate되는가
- 03 — LangChain MeetingBrief Pipeline focused test가 실제로 통과했는가

LangChain MeetingBrief Pipeline 경계 Case

BOUNDARY

provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다.

- 01 — LangChain MeetingBrief Pipeline: raw traceback 대신 stable error_code가 남는가
- 02 — meeting_brief.json: 외부 쓰기·자동 메일이 false인가
- 03 — provider 실패 시 허용 여부에 따라 fallback 또는 구조화 오류가 난다. 재현이 가능한가

LangChain MeetingBrief Pipeline 현업 적용

비용·보안·지연에 따라 로컬과 API 모델을 교체하는 회의 서비스

- 01 — LangChain MeetingBrief Pipeline은 합성·비식별 sample로 먼저 실행
- 02 — 비용·보안·지연에 따라 로컬과 API 모델을 교체하는 회의 서비스의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — meeting_brief.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — meeting_brief.json을 운영 검토 자료로 사용

LangChain MeetingBrief Pipeline 포트폴리오 적용

LangChain MeetingBrief Pipeline · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — OpenAI 오류가 조용히 fixture 성공처럼 보인다.의 사용자 영향을 한 문장으로 설명
- 02 — LangChain MeetingBrief Pipeline 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 새 provider adapter가 기존 MeetingBrief contract와 fallback test를 통과하게 한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — meeting_brief.json과 README 재현 절차를 제출

LangChain MeetingBrief Pipeline 운영 점검

품질

무엇을 Qwen과 fixture가 같은 필수 field를 반환한다.로 정의하는가

복구

OpenAI 오류가 조용히 fixture 성공처럼 보인다. 발생 시 meeting_brief.json에서 원인 상태를 확인

안전

LangChain MeetingBrief Pipeline의 사람 승인과 external_write=false 위치

관측

meeting_brief.json에서 어떤 status·error_code를 보는가

meeting_brief.json 실행 증거

- 01 — 설명: LangChain MeetingBrief Pipeline이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langchain_lab --provider ollama --model qwen3:4b`
- 03 — 증거: meeting_brief.json과 정상·실패 test 결과가 남아 있다.

다음 차시: Action Item 근거 검증

DAY 2 · 7차시

15:50-16:40 (쉬는 시간 17:30-18:00)

Action Item 근거 검증

Evidence · Grounding · validated_meeting_brief.json

진행

Action Item 근거 검증
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/meeting_agent_workflow.py
src/course_services/
meeting_service.py

확인할 결과

validated_meeting_brief.json

7차시 입력과 결과

입력



meeting_brief.json

이번 차시



Action Item 근거 검증

결과



validated_meeting_brief.json

다음 연결



도메인별 Golden Set

Action Item 근거 검증 필요성

담당자와 기한을 잘못 붙이면 회의 자동화가 오히려 조정 비용을 만든다.

결정 추출에서 validated_meeting_brief.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: validated_meeting_brief.json

Action Item 근거 검증 학습 결과

- 01 — 모든 결정과 할 일을 원문 segment ID에 연결한다.
- 02 — evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다. 왜 위험한지 설명한다.
- 03 — validated_meeting_brief.json에서 정상과 실패 상태를 직접 확인한다.

Action Item 근거 검증 용어

용어	이 수업에서 쓰는 뜻
Evidence	결과를 뒷받침하는 원문 구간
Grounding	출력 내용을 입력 근거에 묶는 것
Hallucination	원문에 없는 내용을 만든 오류
Spot check	위험 구간만 사람이 확인

validated_meeting_brief.json 입출력

INPUT

src/meeting_agent_workflow.py

OUTPUT

validated_meeting_brief.json

PROCESS

결정 추출 → Action Item 추출 → evidence ID 연결

VERIFY

모든 evidence ID가 known segment에 존재한다.
없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE
다.

Action Item 근거 검증 실행 흐름

01

결정 추출

02

Action Item 추출

03

evidence ID 연결

04

미상값 null

05

사람 승인

마지막 판단은 `validated_meeting_brief.json`에서 사람이 확인합니다.

validated_meeting_brief.json 정상·실패 계약

정상	——	모든 evidence ID가 known segment에 존재한다.
경계	——	없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
외부 쓰기	——	Action Item 근거 검증: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·validated_meeting_brief.json

Action Item 근거 검증 파일 지도

입력·fixture

src/meeting_agent_workflow.py

구현 코드

src/course_services/meeting_service.py

검증·test

tests/test_meeting_agent_workflow.py

따라하기 notebook

output/day1-meeting-agent-ollama/workflow_result.json

Action Item 근거 검증 개념·코드

Evidence

결과를 뒷받침하는 원문 구간

```
src/meeting_agent_workflow.py
```

Hallucination

원문에 없는 내용을 만든 오류

```
tests/test_meeting_agent_workflow.py
```

Grounding

출력 내용을 입력 근거에 묶는 것

```
src/course_services/meeting_service.py
```

Spot check

위험 구간만 사람이 확인

```
output/day1-meeting-agent-ollama/workflow_result.json
```

Action Item 근거 검증 선택 기준

구분	역할	이번 차시의 판단 기준
Evidence	결정 추출 단계의 입력 기준	결정론 test로 먼저 확인
Grounding	Action Item 추출 단계의 교체 경계	adapter 경계를 확인
Hallucination	evidence ID 연결 단계의 운영 조건	비용·지연·권한을 확인
Spot check	미상값 null 단계의 판단 기록	사람 판단과 운영 기록을 확인

Action Item 근거 검증 대표 실패

실패

evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.

사용자 영향

evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다. 상태에서는 validated_meeting_brief.json을 운영 판단에 사용할 수 없다.

Action Item 근거 검증 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

Action Item 근거 검증 복구 경로

- 01 — 탐지: 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.
- 02 — 중단: evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.
- 03 — 복구: 구조 검사는 code evaluator, 의미 검사는 표본 human review로 나눈다.
- 04 — 증거: validated_meeting_brief.json과 test 결과를 함께 남긴다.

Action Item 근거 검증 Codex 검증 루프

src/course_services/
meeting_service.py의 책임을
찾고
validated_meeting_brief.json
계약을 focused test로 고정합니다.

Action Item 근거 검증
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge

WORKFLOW LAB

Pipeline Review Result

02

LANGGRAPH INTERRUPT

결과를 내보내기 전에 사람이 결정합니다.

REVIEW_REQUIRED

고객 문의 자동화 PoC 범위 회의

배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.

ACTION ITEMS

공개 FAQ 30건 비식별 샘플 정리

서연 · 2026-08-27

근거 s12, s18

응답 JSON Schema와 실패 테스트 작성

준호 · 2026-08-28

근거 s27, s31

interrupt_payload.json

read only

```
{
  "question": "회의 결과를 승인·수정·거절하시겠습니까?",
  "request_id": "meeting-approve-001",
  "summary": "배송 지연과 반품 절차 안내만 1차 자동화 범위로 정하고, 외부 발행 전 사람이 검토한다.",
  "action_items": [
    {
      "task": "공개 FAQ 30건 비식별 샘플 정리",
      "owner": "서연",
      "due_date": "2026-08-27",
      "evidence_ids": [
        "s12",
        "s18"
      ]
    },
    {
      "task": "응답 JSON Schema와 실패 테스트 작성",
      "owner": "준호"
    }
  ]
}
```

Action Item 근거 검증 Codex 작업 명세

목표

Action Item evidence 누락·unknown ID를 각각 재현하는 test를 추가한다.

허용 경로

src/course_services/meeting_service.py
tests/test_meeting_agent_workflow.py

완료 조건

모든 evidence ID가 known segment에 존재한다.
없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

금지

Action Item 근거 검증: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

Action Item 근거 검증 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | Action Item 근거 검증 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 모든 evidence ID가 known segment에 존재한다. |
| 03 | Claude review | —— | validated_meeting_brief.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | Action Item 근거 검증의 두 LLM 의견과 test는 자동 승인이 아니다 |

Action Item 근거 검증 정상 시연

- 01 — 열기: `src/meeting_agent_workflow.py`
- 02 — 구현 확인: `src/course_services/meeting_service.py`
- 03 — 실행: `python -m src.meeting_agent_workflow --help`
- 04 — 성공 신호: 모든 evidence ID가 known segment에 존재한다.

Action Item 근거 검증 실행 명령

```
$ python -m src.meeting_agent_workflow --help
```

실행 전 확인

Action Item 근거 검증: repository root · Python 3.12 · .env 미출력

07_evidence_validation.json 실행 결과

```
{  
  "normal_errors": [],  
  "boundary_errors": [  
    "ACTION_1_UNKNOWN_EVIDENCE:s999"  
  ]  
}
```

Action Item 근거 검증 실패·복구 시연

재현

없는 s99를 가리키면
ACTION_n_UNKNOWN_EVIDENCE다.

복구

구조 검사는 code evaluator, 의미 검사는
표본 human review로 나눈다.

확인: 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

Action Item 근거 검증 소프트웨어 제작

열 파일	<code>src/course_services/meeting_service.py</code> <code>tests/test_meeting_agent_workflow.py</code>
작업 범위	Action Item 근거 검증 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	모든 evidence ID가 known segment에 존재한다. <code>validated_meeting_brief.json</code> 저장

Action Item 근거 검증 Notebook 셀

materials/day2/day2_service_lab.ipynb · 7차시

```
from src.course_services.meeting_service import validate_action_evidence

known_ids = {segment["id"] for segment in stt_segments}
normal_errors = validate_action_evidence(chain_result["result"]["action_items"], known_segment_ids=known_ids)
boundary_errors = validate_action_evidence(
    [{"task": "근거 없는 발행", "evidence_ids": ["s999"]}],
    known_segment_ids=known_ids,
)
evidence_result = {"normal_errors": normal_errors, "boundary_errors": boundary_errors}
assert normal_errors == []
assert boundary_errors == ["ACTION_1_UNKNOWN_EVIDENCE:s999"]
save_json("07_evidence_validation.json", evidence_result)
evidence_result
```

Action Item 근거 검증 Terminal 실행

```
$ python -m src.meeting_agent_workflow --help
```

저장 위치

```
output/course-labs/day2/07_evidence_validation.json
```

모든 evidence ID가 known segment에 존재한다.
실패 시 validated_meeting_brief.json의 status·error_code를 먼저 확인

Action Item 근거 검증 실패 증거

없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

```
result_file = "output/course-labs/day2/07_evidence_validation.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

validated_meeting_brief.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

Action Item 근거 검증 정상 Case

NORMAL

모든 evidence ID가 known segment에 존재한다.

- 01 — src/meeting_agent_workflow.py 입력과 command가 기록됐는가
- 02 — validated_meeting_brief.json schema가 validate되는가
- 03 — Action Item 근거 검증 focused test가 실제로 통과했는가

Action Item 근거 검증 경계 Case

BOUNDARY

없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다.

- 01 — Action Item 근거 검증: raw traceback 대신 stable error_code가 남는가
- 02 — validated_meeting_brief.json: 외부 쓰기·자동 메일이 false인가
- 03 — 없는 s99를 가리키면 ACTION_n_UNKNOWN_EVIDENCE다. 재현이 가능한가

Action Item 근거 검증 현업 적용

회의 후 담당 업무를 자동 제안하되 게시 전 원문 근거를 확인

- 01 — Action Item 근거 검증은 합성·비식별 sample로 먼저 실행
- 02 — 회의 후 담당 업무를 자동 제안하되 게시 전 원문 근거를 확인의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — validated_meeting_brief.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — validated_meeting_brief.json을 운영 검토 자료로 사용

Action Item 근거 검증 포트폴리오 적용

Action Item 근거 검증 · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다.의 사용자 영향을 한 문장으로 설명
- 02 — Action Item 근거 검증 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — Action Item evidence 누락·unknown ID를 각각 재현하는 test를 추가한다.의 diff와 직접 검토한 test 증거를 구분
- 04 — validated_meeting_brief.json과 README 재현 절차를 제출

Action Item 근거 검증 운영 점검

품질

무엇을 모든 evidence ID가 known segment에 존재한다.로 정의하는가

복구

evidence ID가 존재하지만 실제 문장과 의미가 맞지 않는다. 발생 시 validated_meeting_brief.json에서 원인 상태를 확인

안전

Action Item 근거 검증의 사람 승인과 external_write=false 위치

관측

validated_meeting_brief.json에서 어떤 status·error_code를 보는가

validated_meeting_brief.json 실행 증거

- 01 — 설명: Action Item 근거 검증이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.meeting_agent_workflow --help`
- 03 — 증거: validated_meeting_brief.json과 정상·실패 test 결과가 남아 있다.

다음 차시: 도메인별 Golden Set

DAY 2 · 8차시

16:40-17:30 (쉬는 시간 17:30-18:00)

도메인별 Golden Set

Golden case · Evaluator · day2_eval_cases.json

진행

도메인별 Golden Set
강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/
meeting_sample_ko_12min_expected.json
src/observability_lab.py

확인할 결과

day2_eval_cases.json

8차시 입력과 결과

입력 ————— validated_meeting_brief.json

이번 차시 **————— 도메인별 Golden Set**

결과 ————— day2_eval_cases.json

다음 연결 ————— 17:30-18:00 쉬는 시간

도메인별 Golden Set 필요성

범용 요약 점수만으로는 금액·날짜·상품명 같은 도메인 오류를 잡지 못한다.

실패 사례 수집에서 day2_eval_cases.json까지, 가장 먼저 고정할 판단은 무엇인가

이번 차시의 확인 결과: day2_eval_cases.json

도메인별 Golden Set 학습 결과

- 01 — 이커머스·교육·금융 사례에 맞는 평가 case를 만든다.
- 02 — 데모 한 건 성공을 서비스 전체 품질로 일반화한다. 왜 위험한지 설명한다.
- 03 — day2_eval_cases.json에서 정상과 실패 상태를 직접 확인한다.

도메인별 Golden Set 용어

용어	이 수업에서 쓰는 뜻
Golden case	팀이 합의한 대표 입력과 기대 결과
Evaluator	결과를 검사하는 함수
Regression	변경 후 이전 동작이 깨진 상태
Release gate	배포 여부를 가르는 기준

day2_eval_cases.json 입출력

INPUT

data/meeting_sample_ko_12min_expected.json

OUTPUT

day2_eval_cases.json

PROCESS

실패 사례 수집 → 비식별 → expected field 작성

VERIFY

합성 회의 정상 case가 READY_FOR_EXPORT다.
STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

도메인별 Golden Set 실행 흐름

01

실패 사례 수집

02

비식별

03

expected field 작성

04

baseline 실행

05

Day 2 tag

마지막 판단은 `day2_eval_cases.json`에서 사람이 확인합니다.

day2_eval_cases.json 정상·실패 계약

정상	——	합성 회의 정상 case가 READY_FOR_EXPORT다.
경계	——	STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
외부 쓰기	——	도메인별 Golden Set: 기본값 false · dry-run과 사람 승인 뒤 별도 adapter 호출
기록	——	status·error_code·입력 식별자·day2_eval_cases.json

도메인별 Golden Set 파일 지도

입력·fixture

data/meeting_sample_ko_12min_expected.json

구현 코드

src/observability_lab.py

검증·test

tests/test_observability_workflow.py

따라하기 notebook

materials/day2/day2_service_lab.ipynb

도메인별 Golden Set 개념·코드

Golden case

팀이 합의한 대표 입력과 기대 결과

```
data/meeting_sample_ko_12min_expected.json
```

Evaluator

결과를 검사하는 함수

```
src/observability_lab.py
```

Regression

변경 후 이전 동작이 깨진 상태

```
tests/test_observability_workflow.py
```

Release gate

배포 여부를 가르는 기준

```
materials/day2/day2_service_lab.ipynb
```

도메인별 Golden Set 선택 기준

구분	역할	이번 차시의 판단 기준
Golden case	실패 사례 수집 단계의 입력 기준	결정론 test로 먼저 확인
Evaluator	비식별 단계의 교체 경계	adapter 경계를 확인
Regression	expected field 작성 단계의 운영 조건	비용·지연·권한을 확인
Release gate	baseline 실행 단계의 판단 기록	사람 판단과 운영 기록을 확인

도메인별 Golden Set 대표 실패

실패

데모 한 건 성공을 서비스 전체 품질로 일반화한다.

사용자 영향

데모 한 건 성공을 서비스 전체 품질로 일반화한다.
상태에서는 day2_eval_cases.json을 운영
판단에 사용할 수 없다.

도메인별 Golden Set 실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

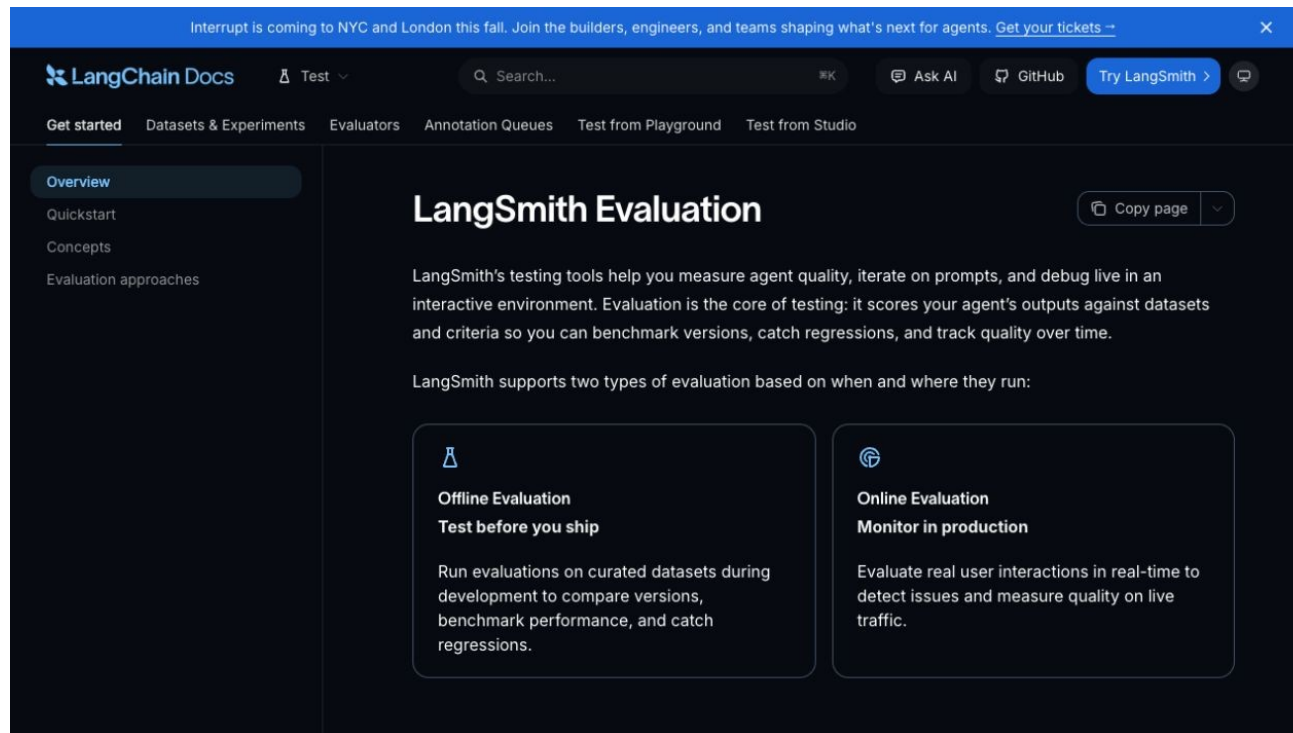
도메인별 Golden Set 복구 경로

- 01 — 탐지: STT HOLD나 사람 거절은 최종 READY가 될 수 없다.
- 02 — 중단: 데모 한 건 성공을 서비스 전체 품질로 일반화한다.
- 03 — 복구: 숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.
- 04 — 증거: day2_eval_cases.json과 test 결과를 함께 남긴다.

도메인별 Golden Set Codex 검증 루프

src/observability_lab.py의
책임을 찾고
day2_eval_cases.json 계약을
focused test로 고정합니다.

도메인별 Golden Set
저장소 이해 → 허용 범위 변경
→ focused test → diff 리뷰
→ 사람 merge



도메인별 Golden Set Codex 작업 명세

목표

대표 실패를 재현하는 test를 먼저 쓰고 최소 변경으로 통과시킨다.

허용 경로

src/observability_lab.py
tests/test_observability_workflow.py

완료 조건

합성 회의 정상 case가 READY_FOR_EXPORT다.
STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

금지

도메인별 Golden Set: secret 출력 · workspace 밖 변경 · test 약화 · 승인 없는 외부 쓰기

도메인별 Golden Set 독립 리뷰

- | | | | |
|----|----------------------|----|---|
| 01 | Codex diff | —— | 도메인별 Golden Set 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 합성 회의 정상 case가 READY_FOR_EXPORT다. |
| 03 | Claude review | —— | day2_eval_cases.json 변경 라인·누락 경계를 별도 session에서 검토 |
| 04 | Human merge | —— | 도메인별 Golden Set의 두 LLM 의견과 test는 자동 승인이 아니다 |

도메인별 Golden Set 정상 시연

- 01 — 열기: `data/meeting_sample_ko_12min_expected.json`
- 02 — 구현 확인: `src/observability_lab.py`
- 03 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py tests/test_observability_workflow.py`
- 04 — 성공 신호: 합성 회의 정상 case가 `READY_FOR_EXPORT`다.

도메인별 Golden Set 실행 명령

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py  
tests/test_observability_workflow.py
```

실행 전 확인

도메인별 Golden Set: repository root · Python 3.12 · .env 미출력

08_day2_scorecard.json 실행 결과

```
{
  "day": 2,
  "service": "Meeting Intelligence Service",
  "status": "SUCCESS",
  "decision": "READY",
  "input": {
    "audio": "data/meeting_sample_ko_12min.wav",
    "transcript": "data/meeting_sample_ko_12min.txt",
    "synthetic": true
  },
  "stages": [
    {
      "name": "audio_contract",
      "status": "READY",
      "artifact": "audio_metadata.json"
    },
    {
      "name": "transcript_segments",
      "status": "SUCCESS",
      "artifact": "transcript.json"
    },
    {
      "name": "evidence_chunks",
      "status": "SUCCESS",
      "artifact": "meeting_chunks.json"
    },
    ...
  ]
}
```

도메인별 Golden Set 실패·복구 시연

재현

STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

복구

숫자·이름·기한·무음·반복을 포함한 작은 golden set부터 운영한다.

확인: STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

도메인별 Golden Set 소프트웨어 제작

열 파일

src/observability_lab.py
tests/test_observability_workflow.py

작업 범위

도메인별 Golden Set 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

합성 회의 정상 case가 READY_FOR_EXPORT다.
day2_eval_cases.json 저장

도메인별 Golden Set Notebook 셀

materials/day2/day2_service_lab.ipynb · 8차시

```
from src.course_services.course_demo import build_course_demo

day2_scorecard = build_course_demo(2, workspace_root=ROOT)
focused_test = run_command(sys.executable, "-m", "pytest", "-q", "tests/test_meeting_agent_workflow.py",
"tests/test_course_services.py")
day2_scorecard["focused_test"] = focused_test
assert day2_scorecard["decision"] == "READY"
assert focused_test["returncode"] == 0
save_json("08_day2_scorecard.json", day2_scorecard)
{"decision": day2_scorecard["decision"], "metrics": day2_scorecard["metrics"]}
```


도메인별 Golden Set Terminal 실행

```
$ python -m pytest -q tests/test_meeting_agent_workflow.py  
tests/test_observability_workflow.py
```

저장 위치

output/course-labs/day2/08_day2_scorecard.json

합성 회의 정상 case가 READY_FOR_EXPORT다.
실패 시 day2_eval_cases.json의 status·error_code를 먼저 확인

도메인별 Golden Set 실패 증거

STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

```
result_file = "output/course-labs/day2/08_day2_scorecard.json"  
expected = "HOLD | EXPECTED_FAILURE | BLOCKED"  
external_write = false
```

day2_eval_cases.json이 HOLD라면 외부 쓰기 없이 해당 차시 입력으로 돌아갑니다.

도메인별 Golden Set 정상 Case

NORMAL

합성 회의 정상 case가 READY_FOR_EXPORT다.

- 01 — data/meeting_sample_ko_12min_expected.json 입력과 command가 기록됐는가
- 02 — day2_eval_cases.json schema가 validate되는가
- 03 — 도메인별 Golden Set focused test가 실제로 통과했는가

도메인별 Golden Set 경계 Case

BOUNDARY

STT HOLD나 사람 거절은 최종 READY가 될 수 없다.

- 01 — 도메인별 Golden Set: raw traceback 대신 stable error_code가 남는가
- 02 — day2_eval_cases.json: 외부 쓰기·자동 메일이 false인가
- 03 — STT HOLD나 사람 거절은 최종 READY가 될 수 없다. 재현이 가능한가

도메인별 Golden Set 현업 적용

현업에서는 상담/회의 유형별 scorecard, 포트폴리오에는 실패 사례 3건

- 01 ——— 도메인별 Golden Set은 합성·비식별 sample로 먼저 실행
- 02 ——— 현업에서는 상담/회의 유형별 scorecard, 포트폴리오에는 실패 사례 3건의 현재 수작업 시간과 오류를 baseline으로 기록
- 03 ——— day2_eval_cases.json 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 ——— day2_eval_cases.json을 운영 검토 자료로 사용

도메인별 Golden Set 포트폴리오 적용

도메인별 Golden Set · 문제·코드·실패·검증을 한 저장소로 제출

- 01 — 데모 한 건 성공을 서비스 전체 품질로 일반화한다.의 사용자 영향을 한 문장으로 설명
- 02 — 도메인별 Golden Set 정상 Demo와 대표 실패 Demo를 함께 준비
- 03 — 대표 실패를 재현하는 test를 먼저 쓰고 최소 변경으로 통과시킨다.의 diff와 직접 검토한 test 증거를 구분
- 04 — day2_eval_cases.json과 README 재현 절차를 제출

도메인별 Golden Set 운영 점검

품질

무엇을 합성 회의 정상 case가 READY_FOR_EXPORT
다.로 정의하는가

복구

데모 한 건 성공을 서비스 전체 품질로 일반화한다. 발생 시
day2_eval_cases.json에서 원인 상태를 확인

안전

도메인별 Golden Set의 사람 승인과
external_write=false 위치

관측

day2_eval_cases.json에서 어떤 status·error_code
를 보는가

day2_eval_cases.json 실행 증거

- 01 — 설명: 도메인별 Golden Set이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_meeting_agent_workflow.py tests/test_observability_workflow.py`
- 03 — 증거: day2_eval_cases.json과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간